Sensor_STOPWATCH_UART_FIFO

작성자 : 홍세헌, 이기광

작성일 : **2026_0820**

version_2

Table of Contents

1. Hardware Design 4
   1.1. Overview 4
      1.1.1. Description 4
      1.1.2. Feature 5
   1.2. Overall block 6
      1.2.1. Block diagram 6
      1.2.2. IO description 7
      1.2.3. Memory list 7
   1.3. Sub block description 8
      1.3.1. Sub block #1 UART_FIFO 9
      1.3.2. Sub block #2 Ascii_decoder 16
      1.3.3. Sub block #3 Controller 19
      1.3.4. Sub block #4 SR04 sensor 23
      1.3.5 Sub block #5 DHT11 sensor 27
      1.3.6 Sub block #6 Ascii_sender 31
2. Simulation 34
   2.1. Top Simulation 34
   2.2. Submodule Simulation 38
      2.2.1. Submodule #1 Tick_generator 38
      2.2.2. Submodule #2 UART_FIFO 38
      2.2.3. Submodule #3 Ascii_decoder 39
      2.2.4. Submodule #4 Control_unit 39
      2.2.5. Submodule #5 SR04_sensor 40
      2.2.6 Submodule #6 DHT11_sensor 40
      2.2.7 Submodule #7 Ascii_sender 41
3. Troubleshooting 42
4. Further work 44
5. Project video 44

1. Hardware Design

1.1. Overview

본 설계는 FPGA에서 동작하는 스톱워치와 센서의 측정 데이터를 UART 통신을 이용하여 PC로 전송하고, PC에서 전달되는 명령을 FPGA가 받아 시스템을 제어할 수 있도록 통합하는 것을 목적으로 하였습니다. 기존에는 스톱워치의 시간, 워치의 시간 또는 거리센서, 온습도 센서의 측정값을 FPGA의 디스플레이를 통해 확인하였습니다. 이 부분에서 UART 통신을 이용하여 FPGA에서 생성되는 데이터를 PC에서도 확인할 수 있도록 하였고, 반대로 PC에서 명령을 전송하여 각 스톱워치, 워치의 실행,정지,초기화 및 옵션기능등의 동작을 제어할 수 있도록 설계하였습니다.



1.1.1. Description

스톱워치, 워치와 센서에서 생성된 데이터는 FIFO에 저장한 후 UART를 통해 순차적으로 전송하며, PC에서 전달되는 명령은 UART 수신부와 데이터 디코더를 통해 해석하여 FPGA의 동작을 제어한다. 이를 통해 FPGA 내부의 데이터를 외부에서 확인하고 제어할 수 있으며, FIFO를 사용하여 서로 다른 데이터 처리 및 UART 전송 속도에 따른 데이터 유실을 방지할 수 있다.

1.1.2. Feature

본 설계는 FPGA를 기반으로 STOPWATCH, WATCH, DHT11 SENSOR, SR04 SENSOR의 4가지 기능을 하나의 시스템으로 통합하고, UART 및 FIFO를 이용하여 외부 PC와 데이터를 송수신할 수 있도록 구현하였다.

1. STOPWATCH

STOPWATCH는 시간의 시작(RUN), 정지(STOP), 초기화(CLEAR), LAP 저장 등의 기능을 제공한다. 100hz 클럭을 기반으로 측정된 시간은 FND를 통해 확인할 수 있도록 구현하였다. 또한 측정된 데이터를 UART를 통해 PC로 전송하여 외부에서도 스톱워치 상태와 시간을 확인할 수 있도록 하였다.

2. WATCH

WATCH는 시, 분, 초 단위의 시간 정보를 지속적으로 표시하는 기능을 수행한다. 시간의 증가 및 자릿수 이동을 제공하며, AM/PM을 구분하여 표시할 수 있도록 구현하였다. 또한 WATCH에서 생성되는 시간 데이터를 UART를 통해 PC로 전송할 수 있도록 구성하였다.

3. DHT11 SENSOR

DHT11 SENSOR는 주변 환경의 온도와 습도 데이터를 측정하는 기능을 수행한다. 센서와의 정해진 통신 타이밍에 따라 데이터를 수신하고, 수신한 온도 및 습도 값을 FPGA에서 처리하여 표시 및 UART 통신을 통해 PC로 전달할 수 있도록 구현하였다.

4. SR04 SENSOR

SR04 SENSOR는 초음파를 이용하여 물체와의 거리를 측정하는 기능을 수행한다. Trigger 신호를 발생시킨 후 Echo 신호의 지속 시간을 측정하고, 이를 거리 값으로 변환하여 표시 및 UART를 통해 PC로 전송할 수 있도록 구현하였다.

5. UART & FIFO

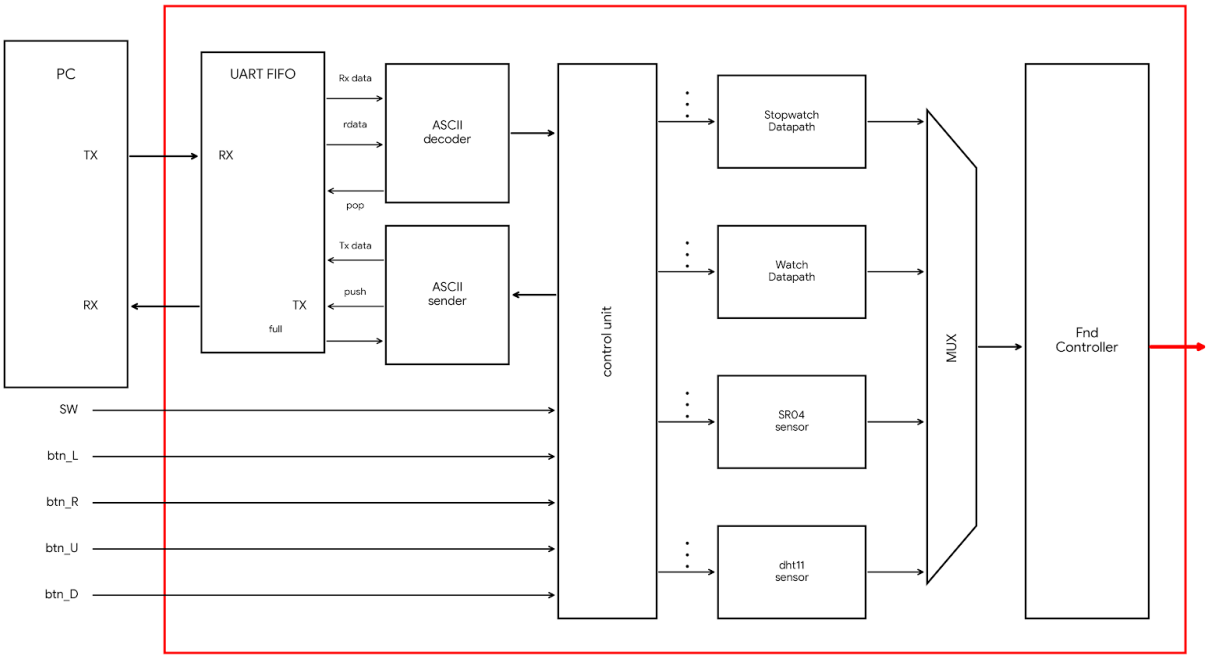
4개의 기능에서 생성되는 데이터를 외부 PC와 주고받기 위해 UART 통신 기능을 적용하였다. PC에서 전달되는 명령은 UART를 통해 수신하여 각 기능의 제어 신호로 변환하고, FPGA에서 생성된 STOPWATCH, WATCH, DHT11, SR04의 데이터는 UART를 통해 PC로 전송한다.

또한 여러 모듈에서 발생하는 데이터와 UART의 전송 속도 차이로 인해 데이터가 유실되는 것을 방지하기 위해 FIFO를 적용하였다. FIFO에 데이터를 저장한 후 UART의 전송 가능 시점에 맞춰 순차적으로 전송함으로써 여러 데이터가 동시에 발생하는 상황에서도 안정적인 데이터 전송이 가능하도록 설계하였다.

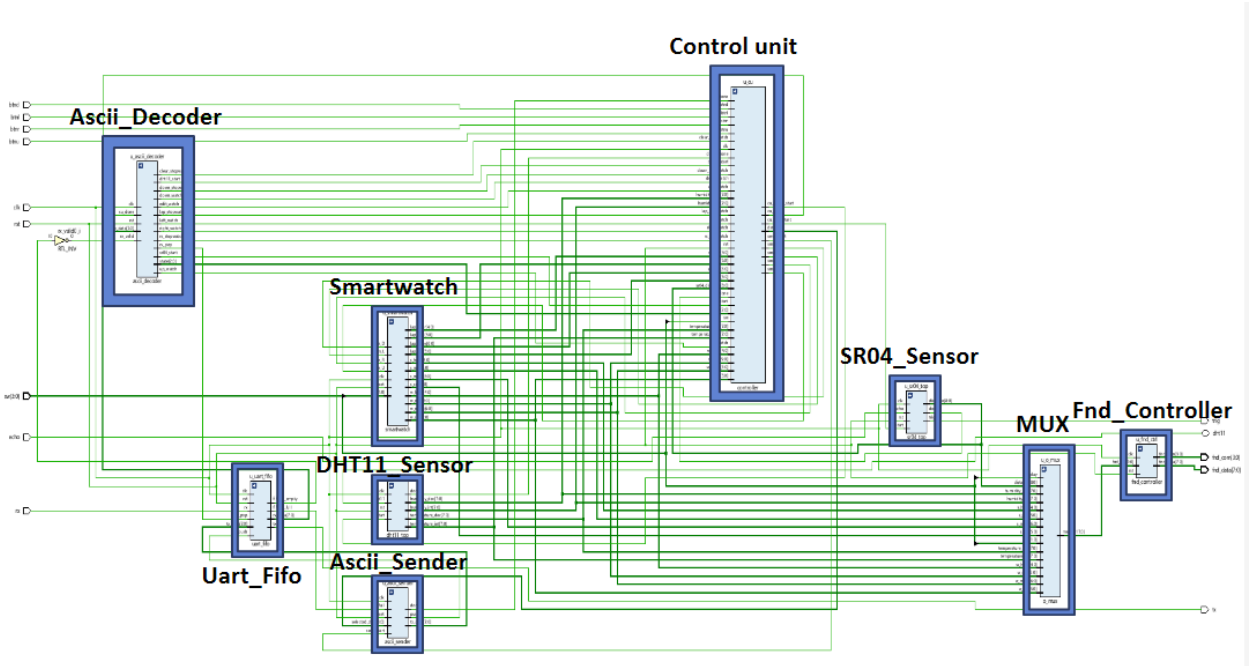
1.2. Overall block

1.2.1. Block diagram

Sensor_STOPWATCH_UART_FIFO



Schematic



1.2.2. IO description

Table 1 Input

Name	Bit width	Initial value	Source Block	Description
clk	1	0	Basys3 board	100MHz clock input
rst	1	0	Basys3 board	Active high reset
rx	1	N/A	Pc to fpga Bridge	UART 수신
echo	1	0	Sr04 sensor	sr04 sensor data 수신
dht11	1	1	Dht11 sensor	dht11 1wire 양방향 통신 라인
sw	4	0	Basys3 board	switch
btnl	1	0	Basys3 board	Left button input
btnr	1	0	Basys3 board	Right button input
btnu	1	0	Basys3 board	Up button input
btnd	1	0	Basys3 board	Down button input

Table 2 Output

Name	Bit width	Initial value	Destination Block	Description
tx	1	N/A	Fpga to pc bridge	UART 송신
trig	1	0	sr04 sensor	거리 측정 트리거
dht11	1	1	dht 11 sensor	dht11 1wire 양방향 통신 라인
fnd_com	4	0	Basys3 FND	4자리 FND 중 활성화할 자리 선택
fnd_data	8	c0	fnd_controller	선택된 자리에 표시할 세그먼트 데이터

1.2.3. Memory list

Table 3 Memory list

Name	Location	Type	Size	Description
register_file	fifo.register_file	BRAM	8*8	data bit width : 8bit Memory depth : 8depth

1.3. Sub block description

UART_FIFO

- uart_controller
    - Baud_tick_gen
    - Uart_tx
    - Uart_rx
- fifo_rx
- fifo_tx

ASCII_DECODER

ASCII_SENDER

- ascii_encoder
- digit_splitter

CONTROLLER

SR04_TOP

- sr04_controller
    - us_tick_sr04

DHT11_TOP

- dht11_controller
    - us_tick_dht11

SMARTWATCH(이전 프로젝트와 같은 내용이므로 검증 생략)

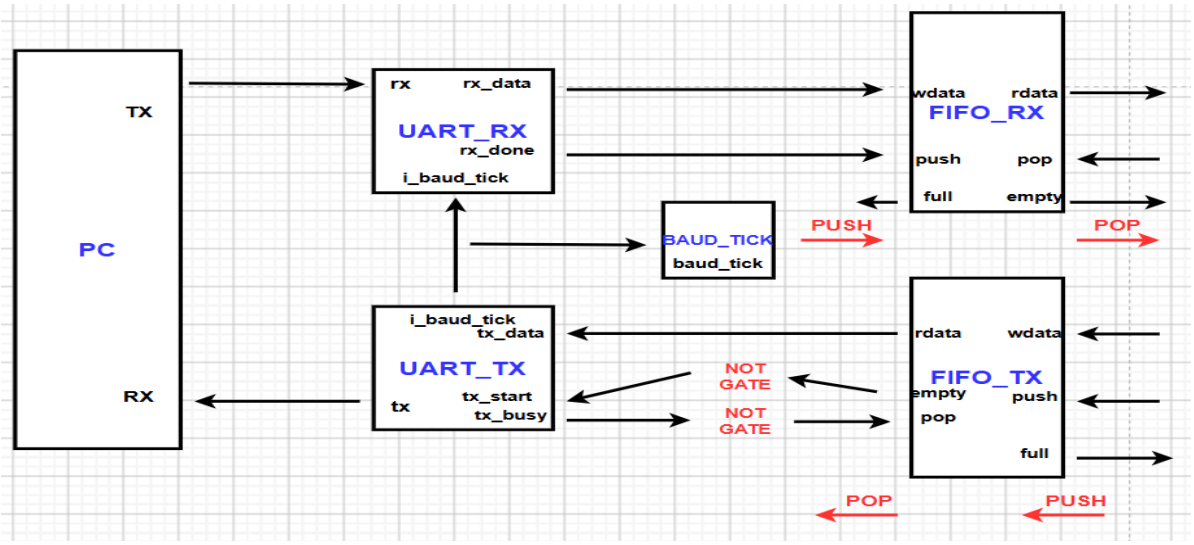
- top_stopwatch
    - stopwatch_datapath
        - time_counter
        - tick_gen_100hz
    - control_unit
    - lap_register
- top_watch
    - watch_datapath
        - tick_gen_100hz
        - watch_counter
    - watch_control_unit

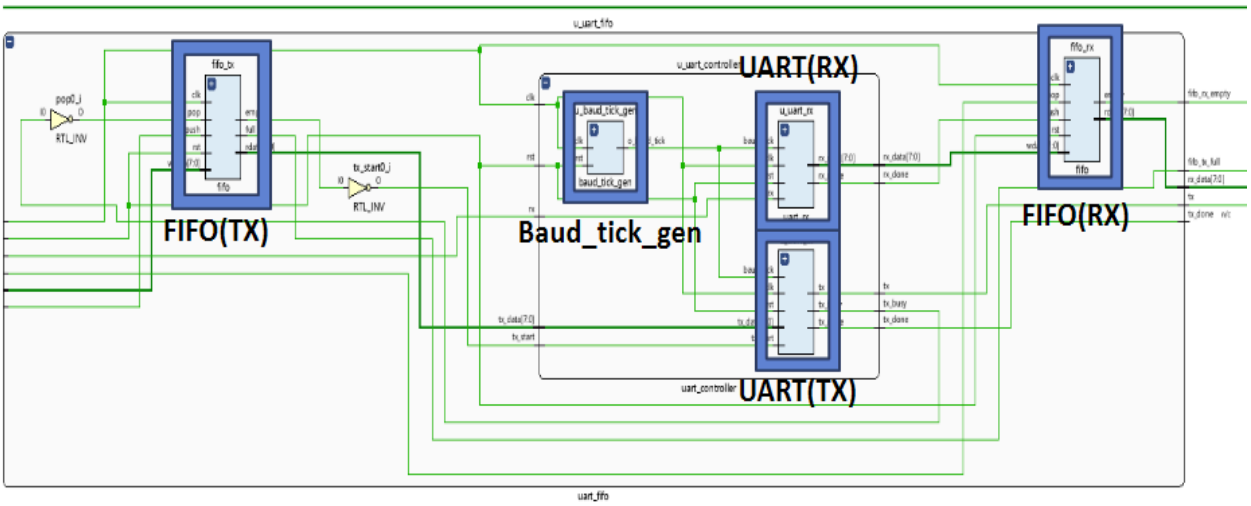
1.3.1.Sub block #1 UART_FIFO

UART_FIFO

UART FIFO는 FPGA와 외부 PC 사이의 UART 통신을 담당하며, 송수신 과정에서 발생하는 데이터를 FIFO에 임시 저장하여 안정적으로 처리하기 위해 설계하였다. 본 시스템에서는 STOPWATCH, WATCH, DHT11, SR04 등 여러 모듈에서 생성되는 데이터를 PC로 전송해야 하므로, 각 모듈에서 발생하는 데이터와 UART의 전송 속도 차이를 고려하여 FIFO 구조를 적용하였다.

Schematic





Baud_tick_generator

Baud Tick Generator는 100 MHz의 시스템 클럭을 UART 통신에 필요한 기준 신호로 변환하는 역할을 수행하였다. 본 설계에서는 9600 bps의 UART 통신을 구현하기 위해 Baud Tick의 발생 주파수를 16배인 153,600 Hz로 설정하였다. 생성된 Baud Tick을 TX와 RX에서 16번씩 카운트하여 하나의 데이터 비트 구간을 구성함으로써 약 9600 bps의 UART 통신 속도를 구현하였다.

코드설명

```
module baud_tick_gen #(
    parameter BAUD_RATE  = 9600,
              CLK_PERIOD = 100_000_000
) (
    input  clk,
    input  rst,
    output o_baud_tick
);

    localparam F_COUNT = CLK_PERIOD / BAUD_RATE;

    reg  [$clog2(F_COUNT)-1:0] baud_cnt_reg;
    wire [$clog2(F_COUNT)-1:0] baud_cnt_next;

    always @(posedge clk, posedge rst) begin
        if (rst) begin
            baud_cnt_reg <= 0;
        end else begin
            baud_cnt_reg <= baud_cnt_next;
        end
    end

    assign baud_cnt_next = (baud_cnt_reg == F_COUNT - 1) ? 0 : baud_cnt_reg + 1;
    assign o_baud_tick   = (baud_cnt_reg == F_COUNT - 1) ? 1 : 0;

endmodule
```

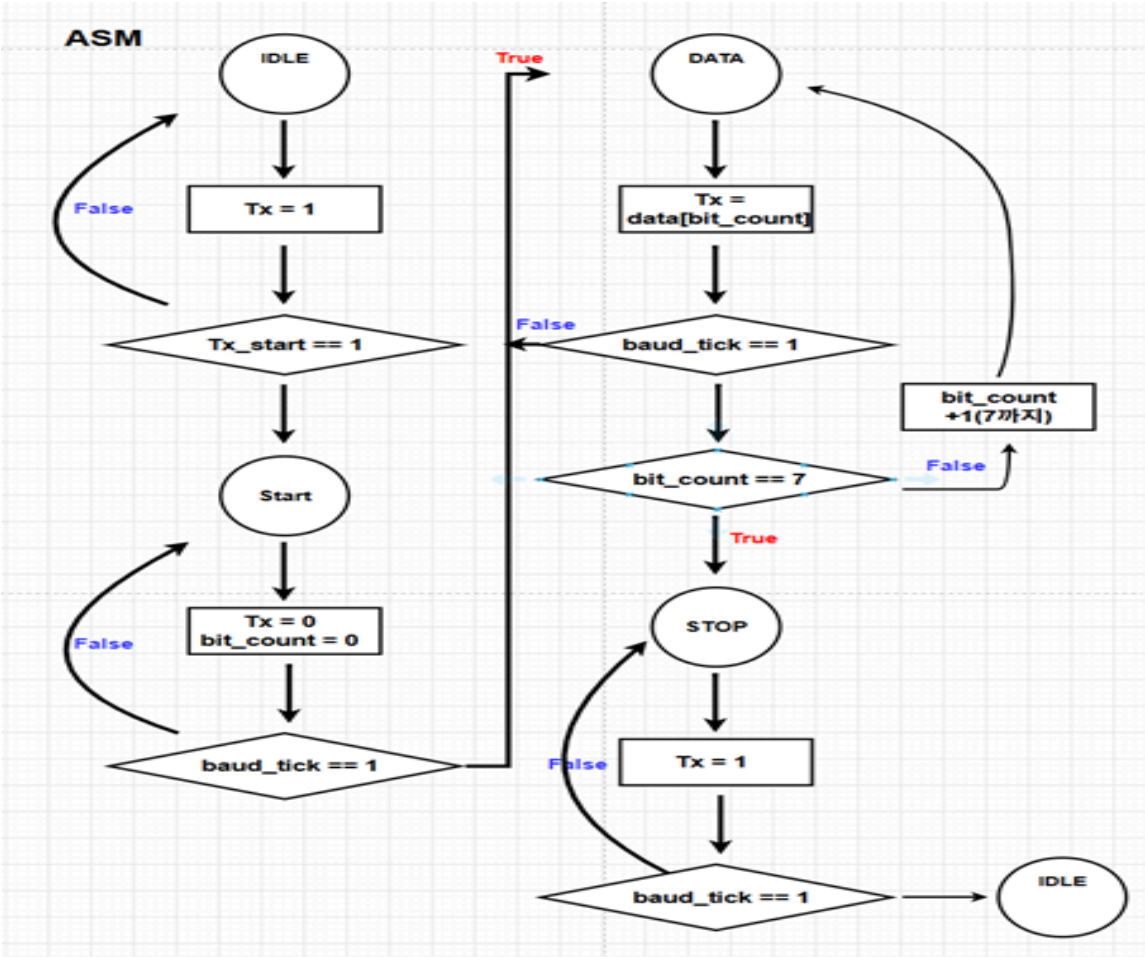
100MHz 메인클럭을 9600bps 통신 타이밍에 맞추기 위한 틱 펄스 생성 (100,000,000 / 9600)

카운트를 활용하여 원하는 주기적인 펄스값 생성

UART(TX)

UART 송신부(TX)는 IDLE, START, DATA, STOP의 4가지 상태로 구성하였다. 전송 요청이 발생하면 송신할 8비트 데이터를 내부 레지스터에 저장하고 START bit, 8비트 DATA, STOP bit 순서로 데이터를 직렬 전송하도록 구현하였다. DATA 전송에서는 저장된 데이터의 LSB부터 순차적으로 출력하도록 구성하였으며, 전송이 완료되면 `tx_done` 신호를 발생시켜 다음 데이터의 전송이 가능하도록 하였다.

UART(TX) ASM



UART(TX) FSM구조 코드설명

```
localparam [1:0] IDLE = 0, START = 1, DATA = 2, STOP = 3;
```

상태값 지정

```
reg [1:0] c_state, n_state;
reg tx_reg, tx_next;
reg tx_busy_reg, tx_busy_next;
reg tx_done_reg, tx_done_next;
reg [2:0] bit_cnt_reg, bit_cnt_next;
reg [3:0] tick_cnt_reg, tick_cnt_next;
reg [7:0] data_reg, data_next;
```

reg값, next값 지정

```
// state register SL
always @(posedge clk, posedge rst) begin
    if (rst) begin
        c_state       <= IDLE;
        bit_cnt_reg   <= 3'd0;
        tx_reg        <= 1'b1;
        data_reg      <= 8'd0;
        tx_busy_reg   <= 1'b0;
        tx_done_reg   <= 1'b0;
        tick_cnt_reg  <= 4'd0;
    end else begin
        c_state       <= n_state;
        bit_cnt_reg   <= bit_cnt_next;
        tx_reg        <= tx_next;
        data_reg      <= data_next;
        tx_busy_reg   <= tx_busy_next;
        tx_done_reg   <= tx_done_next;
        tick_cnt_reg  <= tick_cnt_next;
    end
end
```

현재상태값 지정

next_state 상태 (IDLE > START > DATA > STOP)

```
// next state CL
always @(*) begin
    n_state       = c_state;
    bit_cnt_next  = bit_cnt_reg;
    tx_next       = tx_reg;
    data_next     = data_reg;
    tx_busy_next  = tx_busy_reg;
    tx_done_next  = tx_done_reg;
    tick_cnt_next = tick_cnt_reg;
```

초기값설정
(latch발생하지않음)

```
    case (c_state)
        IDLE: begin
            tx_next       = 1'b1;
            tx_done_next  = 1'b0;
            tick_cnt_next = 4'd0;
            if (tx_start) begin
                tx_busy_next = 1'b1;
                data_next    = tx_data;
                n_state      = START;
            end
        end
```

IDLE

```
        START: begin
            tx_next      = 1'b0;
            bit_cnt_next = 3'd0;
            if (baud_tick) begin
                if (tick_cnt_reg == 15) begin
                    tick_cnt_next = 4'd0;
                    n_state = DATA;
                end else begin
                    tick_cnt_next = tick_cnt_next + 1;
                end
            end
        end
```

START

```
        DATA: begin
            tx_next = data_reg[0];
            if (baud_tick) begin
                if (tick_cnt_reg == 15) begin
                    tick_cnt_next = 4'd0;
                    data_next     = {1'b0, data_reg[7:1]};
                    if (bit_cnt_reg < 7) begin
                        n_state      = DATA;
                        bit_cnt_next = bit_cnt_reg + 1;
                    end else begin
                        n_state = STOP;
                    end
                end else begin
                    tick_cnt_next = tick_cnt_next + 1;
                end
            end
        end
```

DATA

```
        STOP: begin
            tx_next = 1'b1;
            if (baud_tick) begin
                tick_cnt_next = tick_cnt_reg + 1;
                if (tick_cnt_reg == 15) begin
                    tx_busy_next = 1'b0;
                    tx_done_next = 1'b1;  //tick
                    n_state      = IDLE;
                end
            end
        end
    endcase
end

assign tx_busy = tx_busy_reg;
assign tx_done = tx_done_reg;
assign tx      = tx_reg;

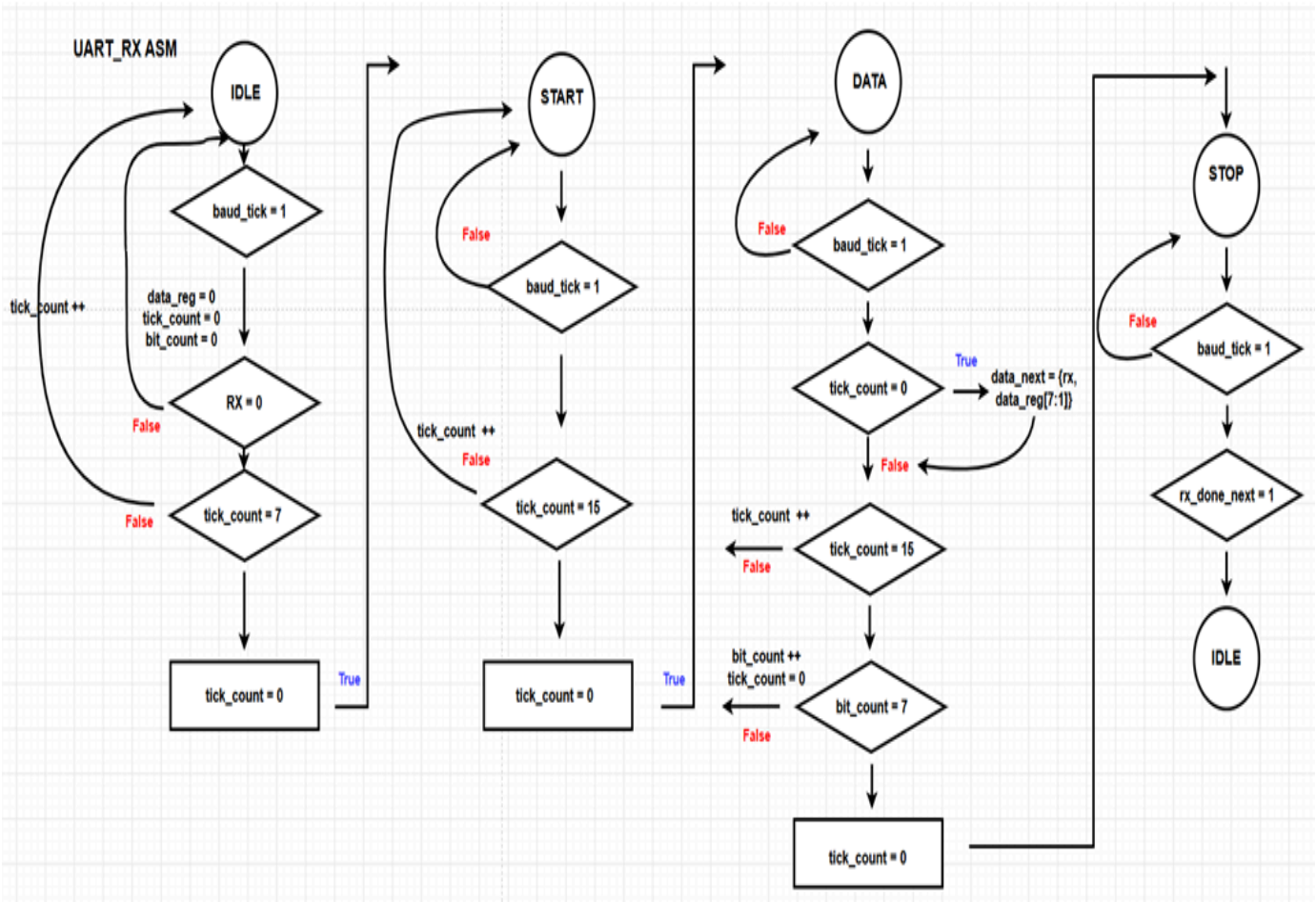
endmodule
```

STOP

UART(RX)

UART 수신부(RX) 역시 IDLE, START, DATA, STOP의 4가지 상태로 구성하였다. IDLE 상태에서 START bit를 감지한 후 데이터 수신을 시작하며, DATA 상태에서 수신된 8비트 데이터를 shift register 방식으로 저장한다. 8비트 데이터의 수신이 완료되고 STOP 상태에 도달하면 `rx_done` 신호를 발생시켜 수신 데이터가 유효함을 알리도록 구현하였다.

UART(RX) ASM



UART(RX) ASM구조 코드설명

```
localparam [1:0] IDLE = 2'd0, START = 2'd1, DATA = 2'd2, STOP = 2'd3;

reg [1:0] c_state, n_state;
reg [7:0] c_rx_data, n_rx_data;
reg [3:0] c_tick_cnt, n_tick_cnt;
reg [2:0] c_bit_cnt, n_bit_cnt;
// for CL output
// reg c_rx_done;
// for SL output
reg c_rx_done, n_rx_done;

// current state SL
always @(posedge clk, posedge rst) begin
    if (rst) begin
        c_state    <= IDLE;
        c_tick_cnt <= 4'd0;
        c_bit_cnt  <= 3'd0;
        c_rx_data  <= 8'd0;
        c_rx_done  <= 1'b0;
    end else begin
        c_state    <= n_state;
        c_tick_cnt <= n_tick_cnt;
        c_bit_cnt  <= n_bit_cnt;
        c_rx_data  <= n_rx_data;
        c_rx_done  <= n_rx_done;
    end
end
```

상태값 지정

현재상태값 지정

next_state 상태 (IDLE > START > DATA > STOP)

```
// next state, output CL
always @(*) begin
    n_state    = c_state;
    n_tick_cnt = c_tick_cnt;
    n_bit_cnt  = c_bit_cnt;
    n_rx_data  = c_rx_data;
    // c_rx_done = 0; // CL
    n_rx_done  = c_rx_done;
    case (c_state)
        IDLE: begin
            // c_rx_done = 0; // CL
            n_rx_done = 1'b0;
            n_bit_cnt = 0;
            if (baud_tick) begin
                if (!rx) begin
                    if (c_tick_cnt == 7) begin
                        n_state    = START;
                        n_tick_cnt = 0;
                    end else begin
                        n_tick_cnt = c_tick_cnt + 1;
                    end
                end else begin
                    // n_rx_data  = 0;
                    n_tick_cnt = 0;
                end
            end
        end
        START: begin
            if (baud_tick) begin
                if (c_tick_cnt == 15) begin
                    n_tick_cnt = 0;
                    n_state    = DATA;
                end else begin
                    n_tick_cnt = c_tick_cnt + 1;
                end
            end
        end
        DATA: begin
            if (baud_tick) begin
                if (c_tick_cnt == 0) begin
                    // n_rx_data = c_rx_data[c_bit_cnt]; // PIPO, bit indexing
                    n_rx_data = {rx, c_rx_data[7:1]};  // SIPO, bitshift
                end
                if (c_tick_cnt == 15) begin
                    n_tick_cnt = 0;
                    if (c_bit_cnt == 7) begin
                        n_state = STOP;
                    end else begin
                        n_bit_cnt = c_bit_cnt + 1;
                    end
                end else begin
                    n_tick_cnt = c_tick_cnt + 1;
                end
            end
        end
        STOP: begin
            if (baud_tick) begin
                // if (c_tick_cnt == 7) begin
                    n_state    = IDLE;
                    n_rx_done = 1'b1;
                    // c_rx_done = 1'b1;
                // end else begin
                    // n_tick_cnt = c_tick_cnt + 1;
                // end
            end
        end
    endcase
end

assign rx_done = c_rx_done;
assign rx_data = c_rx_data;

endmodule
```

초기값설정
(latch발생없앰)

IDLE

START

DATA

STOP

FIFO

FIFO(First In First Out)는 입력된 데이터가 입력 순서대로 출력되는 데이터 저장 구조이다. 본 시스템에서는 여러 기능 모듈에서 생성되는 데이터를 **UART**를 통해 **PC**로 전송하는 과정에서 발생할 수 있는 데이터 처리 속도의 차이를 완충하기 위해 **FIFO**를 적용하였다.

STOPWATCH, WATCH, DHT11, SR04 등의 모듈에서 전송할 데이터가 발생하면 **FIFO**에 순차적으로 저장하고, **UART** 송신부가 데이터를 전송할 수 있는 시점에 **FIFO**에 저장된 데이터를 순서대로 출력하도록 구성하였다. 이를 통해 여러 데이터가 연속적으로 발생하더라도 데이터를 저장하여 안정적으로 전송할 수 있도록 하였다.

또한 **FIFO**의 **`full`**과 **`empty`** 상태를 이용하여 저장 공간의 상태를 확인하였다. **FIFO**가 가득 찬 경우에는 추가 데이터의 입력을 제한하고, **FIFO**가 비어 있는 경우에는 데이터 출력을 수행하지 않도록 구성하였다.

FIFO FSM구조 코드설명

```
module register_file #(
    parameter ADDR_WIDTH = 3,
    BIT_WIDTH = 8
) (
    input                   clk,
    input  [ADDR_WIDTH-1:0] waddr,
    input  [ADDR_WIDTH-1:0] raddr,
    input  [ BIT_WIDTH-1:0] wdata,
    input                   we,
    output [ BIT_WIDTH-1:0] rdata
);

    localparam DEPTH = 2 ** ADDR_WIDTH;
    reg [BIT_WIDTH -1:0] register_file[0:DEPTH-1];

    always @(posedge clk) begin
        if (we) begin
            register_file[waddr] <= wdata;
        end
    end

    //output CL
    assign rdata = register_file[raddr];

endmodule
```

8비트 데이터 폭 사용
3비트 주소값 사용

데이터값 저장

```
module fifo_control_unit #(
    parameter ADDR_WIDTH = 3
) (
    input                   clk,
    input                   rst,
    input                   push,
    input                   pop,
    output [ADDR_WIDTH-1:0] wptr,
    output [ADDR_WIDTH-1:0] rptr,
    output                  full,
    output                  empty
);

    reg [ADDR_WIDTH-1:0] wptr_reg, wptr_next;
    reg [ADDR_WIDTH-1:0] rptr_reg, rptr_next;
    reg full_reg, full_next;
    reg empty_reg, empty_next;

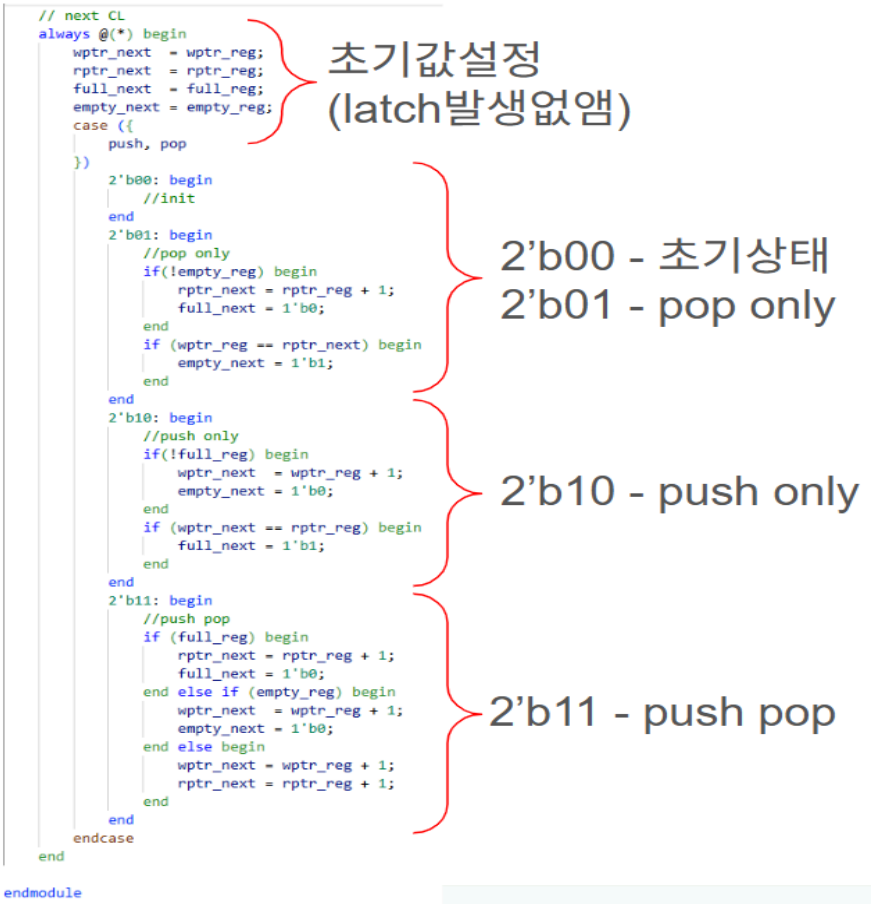
    assign wptr  = wptr_reg;
    assign rptr  = rptr_reg;
    assign full  = full_reg;
    assign empty = empty_reg;

    // state SL
    always @(posedge clk, posedge rst) begin
        if (rst) begin
            wptr_reg  <= 1'b0;
            rptr_reg  <= 1'b0;
            full_reg  <= 1'b0;
            empty_reg <= 1'b1;
        end else begin
            wptr_reg  <= wptr_next;
            rptr_reg  <= rptr_next;
            full_reg  <= full_next;
            empty_reg <= empty_next;

        end
    end
```

reg값, next값 지정

현재상태 지정



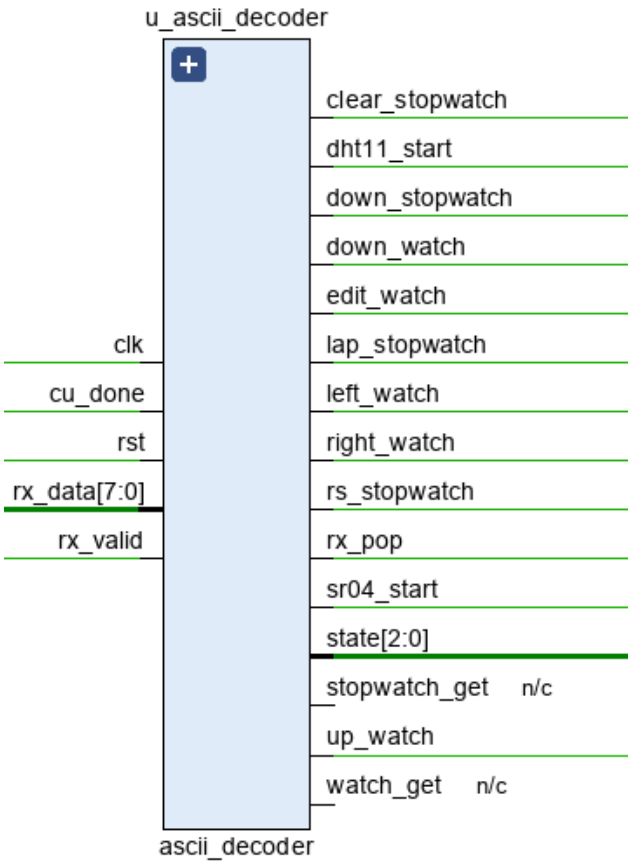

```
// next CL
always @(*) begin
    wptr_next  = wptr_reg;
    rptr_next  = rptr_reg;
    full_next  = full_reg;
    empty_next = empty_reg;
    case ({
        push, pop
    })
        2'b00: begin
            //init
        end
        2'b01: begin
            //pop only
            if(!empty_reg) begin
                rptr_next = rptr_reg + 1;
                full_next = 1'b0;
            end
            if (wptr_reg == rptr_next) begin
                empty_next = 1'b1;
            end
        end
        2'b10: begin
            //push only
            if(!full_reg) begin
                wptr_next  = wptr_reg + 1;
                empty_next = 1'b0;
            end
            if (wptr_next == rptr_reg) begin
                full_next = 1'b1;
            end
        end
        2'b11: begin
            //push pop
            if (full_reg) begin
                rptr_next = rptr_reg + 1;
                full_next = 1'b0;
            end else if (empty_reg) begin
                wptr_next  = wptr_reg + 1;
                empty_next = 1'b0;
            end else begin
                wptr_next = wptr_reg + 1;
                rptr_next = rptr_reg + 1;
            end
        end
    endcase
end

endmodule
```

1.3.2. Sub block #2 Ascii decoder

Ascii decoder 는 **PC**에서 **UART**로 보낸 신호(문자열)을 한 글자씩 받아서, 그게 어떤 명령어인지 알아낸 다음 시스템이 실제로 동작할 수 있는 제어 신호로 바꿔주는 역할을 하는 모듈이다. **UART**로는 바이트 하나하나만 순서대로 들어오기 때문에 그 자체로는 아무 의미가 없는데, 이 모듈이 글자들을 계속 누적해서 어떤 단어가 입력됐는지 판단하고, 그 결과에 따라 지금 **FND**에 어떤 화면을 띄울지 결정하는 **state** 신호와, **dht11**이나 **sr04** 센서를 시작시키는 신호, 그리고 스톱워치나 워치를 조작하는 명령 신호들을 만들어서 내보낸다. 결국 물리적인 버튼이나 스위치가 하던 일을 키보드 입력으로도 똑같이 할 수 있게 중간에서 번역해주는 역할이라고 봐도 무방하다.

Ascii decoder schematic



10진수	16진수	문자	10진수	16진수	문자	10진수	16진수	문자	10진수	16진수	문자
0	0x00	NUL	32	0x20	Space	64	0x40	@	96	0x60	`
1	0x01	SOH	33	0x21	!	65	0x41	A	97	0x61	a
2	0x02	STX	34	0x22	"	66	0x42	B	98	0x62	b
3	0x03	ETX	35	0x23	#	67	0x43	C	99	0x63	c
4	0x04	EOT	36	0x24	$	68	0x44	D	100	0x64	d
5	0x05	ENQ	37	0x25	%	69	0x45	E	101	0x65	e
6	0x06	ACK	38	0x26	&	70	0x46	F	102	0x66	f
7	0x07	BEL	39	0x27	'	71	0x47	G	103	0x67	g
8	0x08	BS	40	0x28	(	72	0x48	H	104	0x68	h
9	0x09	TAB	41	0x29	)	73	0x49	I	105	0x69	i
10	0x0A	LF	42	0x2A	*	74	0x4A	J	106	0x6A	j
11	0x0B	VT	43	0x2B	+	75	0x4B	K	107	0x6B	k
12	0x0C	FF	44	0x2C	,	76	0x4C	L	108	0x6C	l
13	0x0D	CR	45	0x2D	-	77	0x4D	M	109	0x6D	m
14	0x0E	SO	46	0x2E	.	78	0x4E	N	110	0x6E	n
15	0x0F	SI	47	0x2F	/	79	0x4F	O	111	0x6F	o
16	0x10	DLE	48	0x30	0	80	0x50	P	112	0x70	p
17	0x11	DC1	49	0x31	1	81	0x51	Q	113	0x71	q
18	0x12	DC2	50	0x32	2	82	0x52	R	114	0x72	r
19	0x13	DC3	51	0x33	3	83	0x53	S	115	0x73	s
20	0x14	DC4	52	0x34	4	84	0x54	T	116	0x74	t
21	0x15	NAK	53	0x35	5	85	0x55	U	117	0x75	u
22	0x16	SYN	54	0x36	6	86	0x56	V	118	0x76	v
23	0x17	ETB	55	0x37	7	87	0x57	W	119	0x77	w
24	0x18	CAN	56	0x38	8	88	0x58	X	120	0x78	x
25	0x19	EM	57	0x39	9	89	0x59	Y	121	0x79	y
26	0x1A	SUB	58	0x3A	:	90	0x5A	Z	122	0x7A	z
27	0x1B	ESC	59	0x3B	;	91	0x5B	[	123	0x7B	{
28	0x1C	FS	60	0x3C	<	92	0x5C	₩	124	0x7C	|
29	0x1D	GS	61	0x3D	=	93	0x5D	]	125	0x7D	}
30	0x1E	RS	62	0x3E	>	94	0x5E	^	126	0x7E	~
31	0x1F	US	63	0x3F	?	95	0x5F	_	127	0x7F	DEL

(아스키 코드표를 활용하여 명령어에 맞는 동작을 하도록 설계함)

Ascii decoder구조 코드설명

```
assign rx_pop = (rx_valid) ? 1'b1 : 1'b0;

//덧셈 저장, don't touch
always @(posedge clk, posedge rst) begin
    if (rst) begin
        acc_reg <= 0;
        state_reg <= 3'd4;
    end else begin
        if (rx_pop) begin
            acc_reg <= acc_reg + rx_data;
            if (rx_data == 8'h2e) begin
                acc <= acc_reg;
                acc_reg <= 0;
            end
        end
        case(acc_reg)
            32'd681 : begin //sr04
                state_reg <= 3'd0;
            end
            32'd770 : begin //dht 11
                state_reg <= 3'd1;
            end
            32'd887 : begin //watch
                state_reg <= 3'd2;
            end
            32'd669 : begin //stopwatch
                state_reg <= 3'd3;
            end
            default : begin
                state_reg <= 3'd4;
            end
        endcase
    end
end
```

ascii의 총합으로 판별
.(점) 들어오면 종료

case문을 활용하여
acc_reg의 변화를 계속
확인

```
always @(*) begin
    dht11_start_reg = 0;
    sr04_start_reg = 0;
    watch_get_reg = 0;
    stopwatch_get_reg = 0;
    rs_stopwatch_reg = 0;
    clear_stopwatch_reg = 0;
    down_stopwatch_reg = 0;
    lap_stopwatch_reg = 0;
    edit_watch_reg = 0;
    left_watch_reg = 0;
    right_watch_reg = 0;
    up_watch_reg = 0;
    down_watch_reg = 0;
    case(acc)
        32'd770 : begin // get dht11
            if (cnt == 810) begin
                dht11_start_reg = 1'b0;
            end else begin
                dht11_start_reg = 1'b1;
            end
        end
        32'd681 : begin // get sr04
            if (cnt == 810) begin
                sr04_start_reg = 1'b0;
            end else begin
                sr04_start_reg = 1'b1;
            end
        end
        32'd887 : begin // get watch
            if (cnt == 810) begin
                watch_get_reg = 1'b0;
            end else begin
                watch_get_reg = 1'b1;
            end
        end
        32'd669 : begin // get lap
            if (cnt == 810) begin
                stopwatch_get_reg = 1'b0;
            end else begin
                stopwatch_get_reg = 1'b1;
            end
        end
        32'd1250 : begin // rs stopwatch
            if (cnt == 810) begin
                rs_stopwatch_reg = 1'b0;
            end else begin
                rs_stopwatch_reg = 1'b1;
            end
        end
```

초기값지정(latch발생막음)

dht11센서값
불러오기

sr04센서값
불러오기

watch값 불러오기

stopwatch값
불러오기

run stopwatch /
stop stopwatch

```
        32'd1540 : begin // clear stopwatch
            if (cnt == 810) begin
                clear_stopwatch_reg = 1'b0;
            end else begin
                clear_stopwatch_reg = 1'b1;
            end
        end
        32'd1461 : begin // down stopwatch
            if (cnt == 810) begin
                down_stopwatch_reg = 1'b0;
            end else begin
                down_stopwatch_reg = 1'b1;
            end
        end
        32'd1338 : begin // lap stopwatch
            if (cnt == 810) begin
                lap_stopwatch_reg = 1'b0;
            end else begin
                lap_stopwatch_reg = 1'b1;
            end
        end
        32'd989 : begin // edit watch
            if (cnt == 810) begin
                edit_watch_reg = 1'b0;
            end else begin
                edit_watch_reg = 1'b1;
            end
        end
        32'd994 : begin // left watch
            if (cnt == 810) begin
                left_watch_reg = 1'b0;
            end else begin
                left_watch_reg = 1'b1;
            end
        end
        32'd1109 : begin // right watch
            if (cnt == 810) begin
                right_watch_reg = 1'b0;
            end else begin
                right_watch_reg = 1'b1;
            end
        end
```

Stopwatch
값 초기화

Stopwatch
시간감소

Stopwatch
값 저장

watch값
초기화

watch
자릿수이동

watch
자릿수이동

```
        32'd796 : begin // up watch
            if (cnt == 810) begin
                up_watch_reg = 1'b0;
            end else begin
                up_watch_reg = 1'b1;
            end
        end
        32'd1007 : begin // down watch
            if (cnt == 810) begin
                down_watch_reg = 1'b0;
            end else begin
                down_watch_reg = 1'b1;
            end
        end
    endcase
end
// output
assign dht11_start = dht11_start_reg;
assign sr04_start = sr04_start_reg;
assign watch_get = watch_get_reg;
assign stopwatch_get = stopwatch_get_reg;
assign state = state_reg;

assign rs_stopwatch = rs_stopwatch_reg;
assign clear_stopwatch = clear_stopwatch_reg;
assign down_stopwatch = down_stopwatch_reg;
assign lap_stopwatch = lap_stopwatch_reg;
assign edit_watch = edit_watch_reg;
assign left_watch = left_watch_reg;
assign right_watch = right_watch_reg;
assign up_watch = up_watch_reg;
assign down_watch = down_watch_reg;

// butten debounce
always @(posedge clk, posedge rst) begin
    if (rst) begin
        cnt <= 0;
        // r_rx_done <= 0;
    end else begin
        if (rx_valid) begin
            cnt <= 0;
        end else if (cnt == 810) begin
            cnt <= cnt;
        end else begin
            cnt <= cnt +1;
        end
    end
end
```

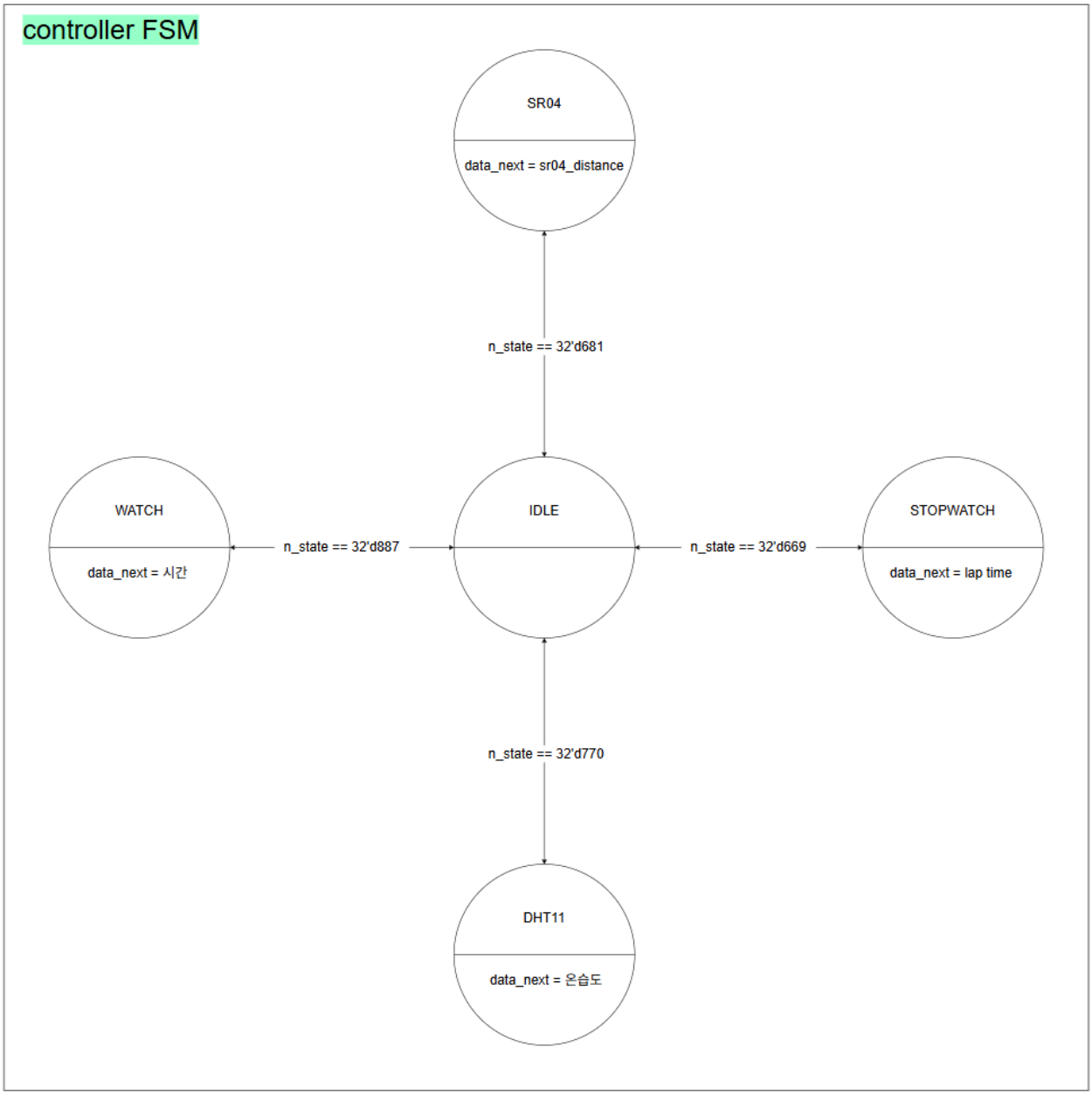
watch
시간증가

watch
시간감소

output 선언

btn_debounce
: 8.1us로 맞춤

1.3.3.Sub block #3 controller



IDLE 상태에서 **ascii_decoder** 로 부터 상태값을 받아와 **FSM**의 천이가 이루어진다.

```
always @(posedge clk, posedge rst) begin
    if (rst) begin
        c_state <= IDLE;
        data_reg <= 0;
        cnt_reg <= 0;
        oe_reg <= 0;
        send_start_reg <= 0;
        wait_done_reg <= 0;
    end else begin
        c_state        <= n_state;
        data_reg       <= data_next;
        cnt_reg        <= cnt_next;
        oe_reg         <= oe_next;
        send_start_reg <= send_start_next;
        wait_done_reg  <= wait_done_next;
    end
end
```

State updata SL

```
always @(*) begin
    n_state = c_state;
    data_next = data_reg;
    cnt_next = cnt_reg;
    oe_next = oe_reg;
    send_start_next = send_start_reg;
    wait_done_next = wait_done_reg;
    w_hour_reg = w_hour;
    w_min_reg = w_min;
    w_sec_reg = w_sec;
    w_msec_reg = w_msec;
    s_hour_reg = s_hour;
    s_min_reg = s_min;
    s_sec_reg = s_sec;
    s_msec_reg = s_msec;

    case (c_state)
        IDLE: begin
            n_state = state;
        end

        SR04: begin
            if (!oe_reg) begin
                if (sr04_done) begin
                    oe_next        = 1'b1;
                    cnt_next       = 0;
                    wait_done_next = 1'b0;
                end
            end else begin
                if (!wait_done_reg && !send_start_reg) begin
                    data_next      = sr04_distance;
                    send_start_next = 1'b1;
                    wait_done_next  = 1'b1;
                end else if (send_start_reg) begin
                    send_start_next = 1'b0;  // 펄스 1클럭만 유지
                end else if (wait_done_reg && ascii_done) begin
                    n_state        = IDLE;
                    oe_next        = 1'b0;
                    wait_done_next = 1'b0;
                end
            end
        end
```

Latch 방지 블록

IDLE : state 값에 따라 n_state 업데이트

SR04:

1. 센서 완료(sr04_done) 대기 후 send 시작
2. 거리 데이터 1개 전송(send_start)
3. Ascii_done 응답 후 IDLE 복귀

```
DHT11: begin
    if (!oe_reg) begin
        if (dht11_done) begin
            oe_next        = 1'b1;
            cnt_next       = 0;
            wait_done_next = 1'b0;
        end
    end else begin
        if (!wait_done_reg && !send_start_reg) begin
            case (cnt_reg)
                4'd0: data_next = temperature_int;
                4'd1: data_next = temperature_dec;
                4'd2: data_next = humidity_int;
                4'd3: data_next = humidity_dec;
            endcase
            send_start_next = 1'b1;
            wait_done_next  = 1'b1;
        end else if (send_start_reg) begin
            send_start_next = 1'b0;
        end else if (wait_done_reg && ascii_done) begin
            if (cnt_reg == 4'd3) begin
                n_state        = IDLE;
                oe_next        = 1'b0;
                cnt_next       = 0;
                wait_done_next = 1'b0;
            end else begin
                cnt_next       = cnt_reg + 1;
                wait_done_next = 1'b0;
            end
        end
    end
end
```

DHT11 :

1. 센서 완료(dht11_done) 대기 후 send 시작
2. 온습도 데이터 4개를 cnt_reg기준 순차 전송
3. 4개 전송시 IDLE 복귀

```
WATCH: begin
    if (!oe_reg) begin
        w_hour_reg     = w_hour;
        w_min_reg      = w_min;
        w_sec_reg      = w_sec;
        w_msec_reg     = w_msec;
        oe_next        = 1'b1;
        cnt_next       = 0;
        wait_done_next = 1'b0;
    end else begin
        if (!wait_done_reg && !send_start_reg) begin
            case (cnt_reg)
                4'd0: data_next = w_hour_reg;
                4'd1: data_next = w_min_reg;
                4'd2: data_next = w_sec_reg;
                4'd3: data_next = w_msec_reg;
            endcase
            send_start_next = 1'b1;
            wait_done_next  = 1'b1;
        end else if (send_start_reg) begin
            send_start_next = 1'b0;
        end else if (wait_done_reg && ascii_done) begin
            if (cnt_reg == 4'd3) begin
                n_state        = IDLE;
                oe_next        = 1'b0;
                cnt_next       = 0;
                wait_done_next = 1'b0;
            end else begin
                cnt_next       = cnt_reg + 1;
                wait_done_next = 1'b0;
            end
        end
    end
end
```

WATCH :

1. 시간,분,초,밀리초 값을 캡처 후 oe 진입
2. 시각 데이터 4개를 cnt_reg기준 순차 전송
3. 4개 전송시 IDLE 복귀

```
STOPWATCH: begin
    if (!oe_reg) begin
        s_hour_reg     = s_hour;
        s_min_reg      = s_min;
        s_sec_reg      = s_sec;
        s_msec_reg     = s_msec;
        oe_next        = 1'b1;
        cnt_next       = 0;
        wait_done_next = 1'b0;
    end else begin
        if (!wait_done_reg && !send_start_reg) begin
            case (cnt_reg)
                4'd0: data_next = s_hour_reg;
                4'd1: data_next = s_min_reg;
                4'd2: data_next = s_sec_reg;
                4'd3: data_next = s_msec_reg;
            endcase
            send_start_next = 1'b1;
            wait_done_next  = 1'b1;
        end else if (send_start_reg) begin
            send_start_next = 1'b0;
        end else if (wait_done_reg && ascii_done) begin
            if (cnt_reg == 4'd3) begin
                n_state        = IDLE;
                oe_next        = 1'b0;
                cnt_next       = 0;
                wait_done_next = 1'b0;
            end else begin
                cnt_next       = cnt_reg + 1;
                wait_done_next = 1'b0;
            end
        end
    end
end
```

STOPWATCH :

1. Lap한 시각 값을 캡처 후 oe 진입
2. 시각 데이터 4개를 cnt_reg기준 순차 전송
3. 4개 전송시 IDLE 복귀

FSM 구조

Controller는 **4**가지 서로 다른 작업**(sr04** 거리측정**, dht11**온습도 측정**, watch**의 시각정보**, stopwatch**의 **laptime)**을 처리해야하는데**,** 각 작업마다 보내야할 데이터의 개수가 다르다**.**

따라서 설계는 상태를 두 계층으로 분리하여 진행되었다**.**

첫번째로 지금 어떤 작업중인가를 관리한다**.(IDLE, SR04, DHT11, WATCH, STOPWATCH)**

두번째로 작업내에서 데이터의 개수**(cnt_reg)**만큼 데이터를 내보낸다**.**

즉 현재 작업을 그대로 두고 내부 카운터를 두어 해당 작업의 데이터 개수만큼 내보낼 수 있도록 처리하였다**.**

Ascii sender와 **handshake** 구조를 적용했다**.** 초기 **ascii sender**가 데이터 한 개를 처리하는 데 걸리는 클럭 수를 미리 계산해서**,** 그 클럭수 만큼 카운터를 세고 다음 데이터를 넘기는 방식을 고려했다**.** 이 경우 코드가 길어져 가독성이 떨어지며 또한 **ascii sender**의 데이터 처리 타이밍이 고정되어있다면 문제가 되지않지만 데이터 처리의 유연함이 떨어진다는 단점이 있었다**.**

이를 개선하기 위해 **ascii_sender**가 데이터 처리를 완료했다는 **done** 신호를 받을 때까지 대기하는 구조로 변경하여 **handshake** 방식을 채택했다**.** 완료 상태를 기다림으로써 **ascii_sender**가 한개의 데이터를 다 처리하지 않은 상태에서 다음 데이터를 내보내는 일을 방지하였다**.**

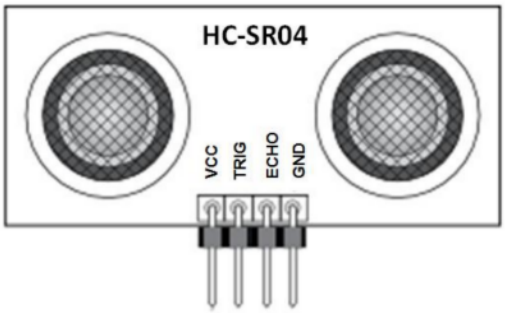
출력 데이터는 **8bit**가 아닌 **9bit**으로 설정하였는데 **sr04**의 거리데이터가 **9bit**으로 **fpga**에 입력되어 출력신호를 **9bit**으로 설정하였다**.**

1.3.4.Sub block #4 sr04 sensor

SR04는 초음파를 이용해 거리를 측정한다. 초음파를 쏘고, 그게 물체에 부딪혀 돌아오는 시간을 재서 "얼마나 멀리 있는지"를 계산하는 방식이다.

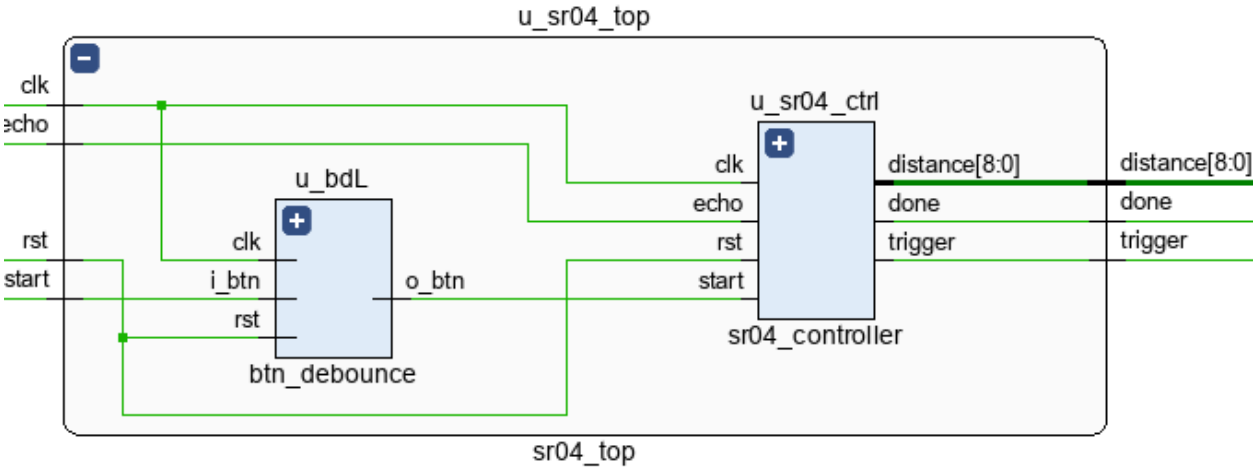
겉을 보면 원통 두 개가 나란히 붙어 있는데, 하나는 송신용(TX, 소리를 쏘는 쪽), 하나는 수신용(RX, 반사파를 듣는 쪽)이다. 이 소재는 전기를 주면 진동해서 초음파를 만들어내고, 반대로 초음파가 부딪히면 전기 신호를 만들어내는 양방향 특성을 가지고 있다. 그래서 송신 쪽은 전기→소리, 수신 쪽은 소리→전기로 변환하는 데 각각 이 소재를 사용한다.

SR04 SENSOR

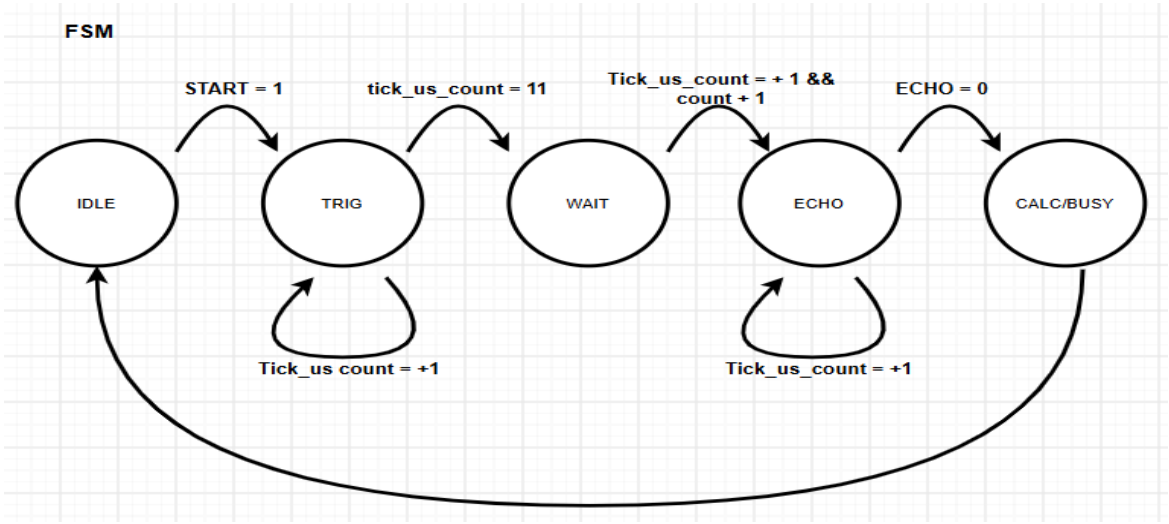


VCC	3.3v ~ 5V
TRIG	Triggering Input Pin. 10uS TTL Pulses
ECHO	TTL Logic Output Pin. Proportional to distance
GND	Ground Pin

SR04 Schematic



SR04 FSM구조



SR04 TIMING DIAGRAM

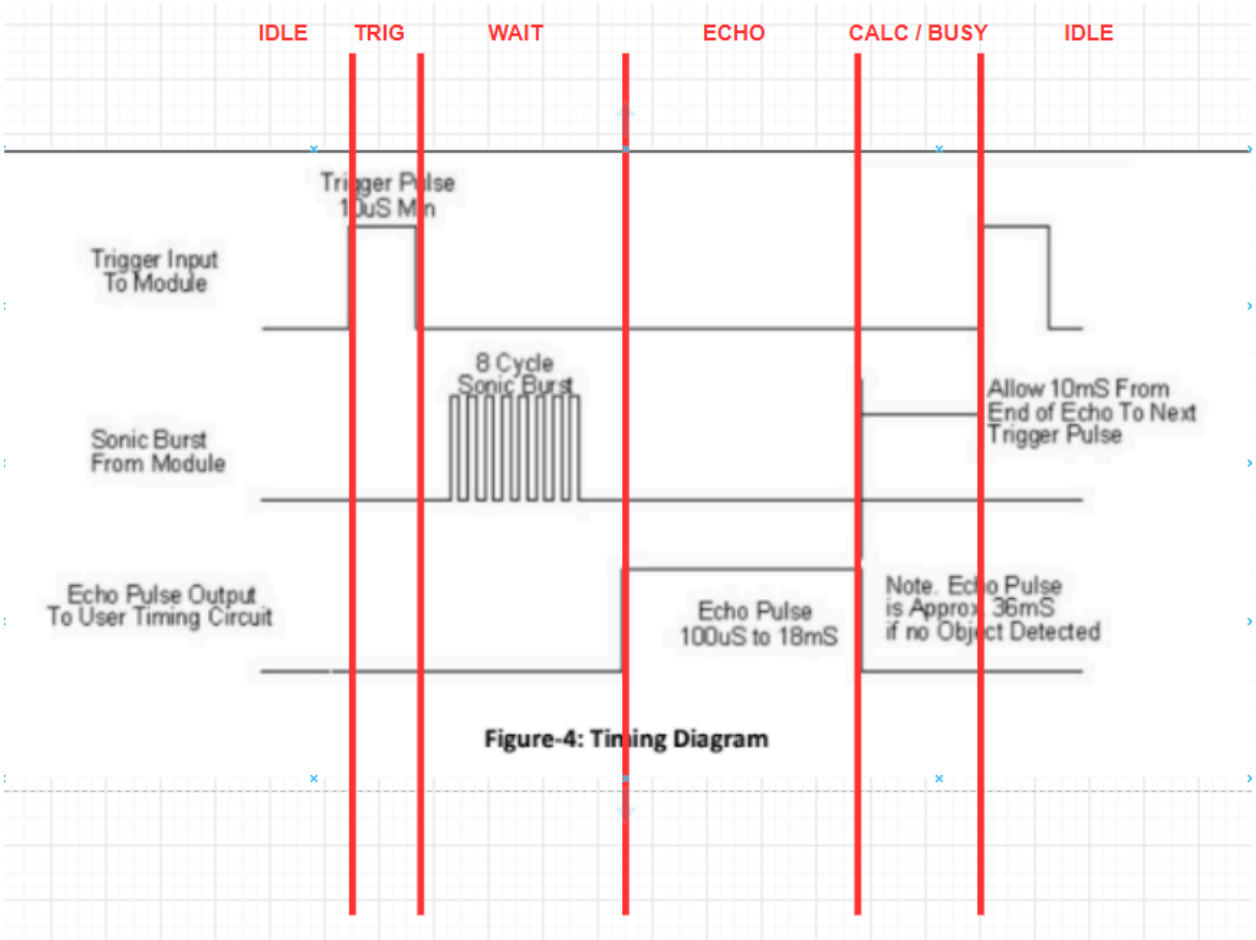


Figure-4: Timing Diagram

TIMING DIAGRAM 단계

IDLE

`trigger`와 done신호가 0으로 내려간 채 `start` 신호만 기다리는 상태이다. IDLE에서 START로 들어오면 카운터를 0으로 초기화하고 TRIG로 넘어간다. 타이밍 다이어그램 그림에서 맨 위에 파형이 해당된다.

TRIG

`tick_us`가 뜰 때마다(=1us마다) 카운터를 올리면서 `trigger`를 계속 1로 유지한다. 11번(≈11us) 세면 trigger를 다시 0으로 내리고 WAIT으로 넘어간다. 데이터시트가 "Trigger Pulse 10uS Min"이라고 명시한 걸 정확히 만족시키기 위해 1us의 마진을 남겼다.

WAIT

Echo 신호가 1로 올라올 때까지 그냥 기다리는 상태이다. 데이터시트 그림에서 "8 Cycle Sonic Burst"(센서가 실제로 40KHz 초음파를 쏘는 구간)이 바로 이 WAIT 구간 동안 물리적으로 일어나고 있다. echo가 1이 되면 반사되어 돌아오는 신호라는 뜻이라 ECHO로 넘어간다.

ECHO

echo가 1인 동안 계속 `us_tick_cnt_reg`를 1씩 증가한다. ("Echo Pulse 100uS to 18mS" 구간의 실제 폭(us)을 재는 부분) echo가 0으로 떨어지는 순간 측정 끝나고 CALC로 넘어간다. 18000(=18ms)이 넘으면 무효 처리하는 것도 데이터시트의 "물체가 없으면 echo가 약 36ms까지 유지됨"이라는 조건을 반영함

CALC

ECHO 종료 후 `Formula` 거리 계산을 수행하기 위해 추가한 상태이다. 기존 busy에 10ms + 마진 1ms를 두어 11ms로 구현하려고 했으나 단순 `/58` 연산에서 발생한 타이밍 이슈(나눗셈을 이용하여 회로 경로가 길어짐)를 해결하기 위해 감산기와 가산기를 이용한 반복 연산 방식으로 구현하였다. 최대 400cm 측정을 고려하여 약 400 Cycle이 필요하므로, 충분한 여유를 두고 1ms의 계산 시간을 확보하였다.

BUSY

10ms(10000us) 동안 그냥 대기하다가 done을 1로 띄우고 IDLE로 복귀한다. 다이어그램에서 "Allow 10ms From End of Echo To Next Trigger Pulse"라고 정확히 이 값(10ms)을 명시하고 있기 때문이다.

SR04 sender구조 코드설명

```
parameter [2:0] IDLE = 0, TRIG = 1, WAIT = 2, ECHO = 3, CALC = 4, BUSY = 5;

reg [2:0] c_state, n_state;
reg [15:0] us_tick_cnt_reg,us_tick_cnt_next;
reg [15:0] calc_reg, calc_next;
reg [8:0] calc_distance_reg, calc_distance_next;
reg [8:0] distance_reg, distance_next;
reg done_reg, done_next, trigger_reg, trigger_next;

wire clear; // to us_tick_gen
wire w_tick_us;

always @(posedge clk, posedge rst) begin
    if (rst) begin
        c_state <= IDLE;
        us_tick_cnt_reg <= 0;
        distance_reg <= 0;
        done_reg <= 0;
        trigger_reg <= 0;
        calc_reg <= 0;
        calc_distance_reg <= 0;
    end else begin
        c_state <= n_state;
        us_tick_cnt_reg <= us_tick_cnt_next;
        distance_reg <= distance_next;
        done_reg <= done_next;
        trigger_reg <= trigger_next;
        calc_reg <= calc_next;
        calc_distance_reg <= calc_distance_next;
    end
end
```

상태 지정

현재상태 지정

```
always @(*) begin
    n_state = c_state;
    us_tick_cnt_next = us_tick_cnt_reg;
    distance_next = distance_reg;
    done_next = done_reg;
    trigger_next = trigger_reg;
    calc_next = calc_reg;
    calc_distance_next = calc_distance_reg;
    case(c_state)
        IDLE : begin
            done_next = 1'b0;
            if(start) begin
                n_state = TRIG;
                us_tick_cnt_next = 0;
            end
        end

        TRIG : begin
            if (tick_us) begin
                if (us_tick_cnt_reg == 11) begin
                    n_state = WAIT;
                    trigger_next = 1'b0;
                    us_tick_cnt_next = 0;
                end else begin
                    trigger_next = 1'b1;
                    us_tick_cnt_next = us_tick_cnt_reg + 1;
                end
            end
        end

        WAIT : begin
            if(tick_us) begin
                if(echo == 1'b1) begin
                    n_state = ECHO;
                    us_tick_cnt_next = us_tick_cnt_reg + 1;
                end
            end
        end
```

초기값 지정

IDLE

TRIG

WAIT

```
        ECHO : begin
            if(tick_us) begin
                if(echo == 1'b0) begin
                    n_state = CALC;
                    us_tick_cnt_next = 0;
                    if (us_tick_cnt_reg <= 18000) begin // 18ms
                        calc_next = us_tick_cnt_reg;
                    end else begin
                        calc_next = 0;
                    end
                end else begin
                    us_tick_cnt_next = us_tick_cnt_reg + 1;
                end
            end
        end

        CALC : begin
            if(tick_us) begin
                if (us_tick_cnt_reg == 1000 -1) begin
                    n_state = BUSY;
                    us_tick_cnt_next = 0;
                    distance_next = calc_distance_reg;
                end else begin
                    us_tick_cnt_next = us_tick_cnt_reg + 1;
                    calc_distance_next = calc_reg / 58;
                end
            end
        end

        BUSY : begin
            if(tick_us) begin
                if (us_tick_cnt_reg == 10000 -1) begin
                    n_state = IDLE;
                    us_tick_cnt_next = 0;
                    done_next = 1'b1;
                end else begin
                    us_tick_cnt_next = us_tick_cnt_reg + 1;
                end
            end
        end
    endcase
end
```

ECHO

CALC

BUSY

```
    assign clear = (c_state == IDLE) ? 1'b1:1'b0;
    assign distance = distance_reg;
    assign done = done_reg;
    assign trigger = trigger_reg;

us_tick_sr04 u_us_tick(
    .clk(clk),
    .rst(rst),
    .run_stop(1'b1),
    .clear(clear),
    .tick_us(tick_us)
);

endmodule

module us_tick_sr04 (
    input clk,
    input rst,
    input run_stop,
    input clear,
    output tick_us
);

reg [6:0] tick_cnt;

always @(posedge clk, posedge rst, posedge clear) begin
    if(rst | clear) begin
        tick_cnt <= 0;
    end else begin
        if (run_stop) begin
            if (tick_cnt == (100 - 1)) begin
                tick_cnt <= 0;
            end else begin
                tick_cnt <= tick_cnt + 1;
            end
        end else begin
            tick_cnt <= tick_cnt;
        end
    end
end

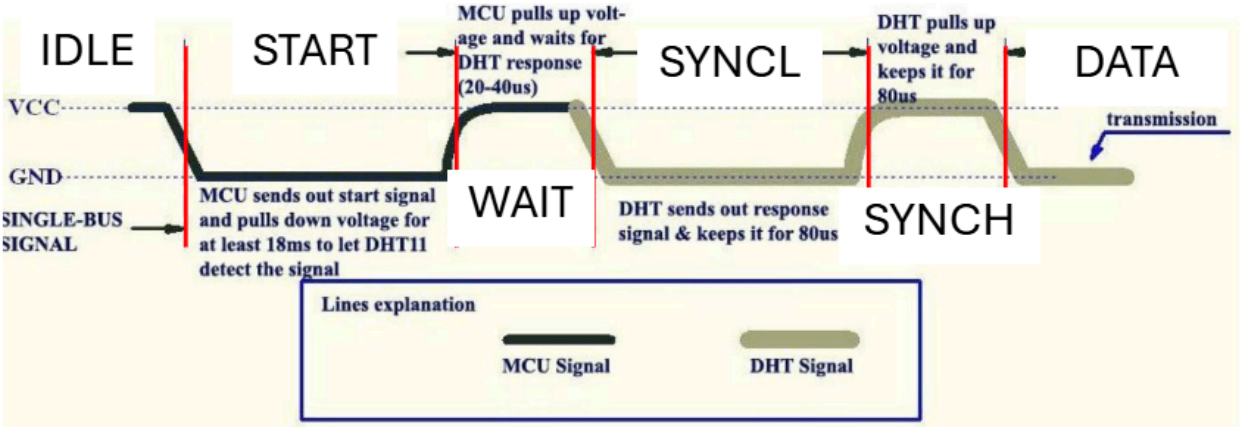
assign tick_us = (tick_cnt == (100 -1)) ? 1'b1 : 1'b0;
endmodule
```

tick_sr04 instance

1us tick 펄스

1.3.5.Sub block #5 dht11

dht11 top 모듈의 내부는 button debounce 모듈과 dht11 control 모듈이 있다. Dht11 control 모듈은 FSM 으로 구현하였고 Button debounce 모듈은 생략하고 sub block 설명을 진행한다.
사전 지식

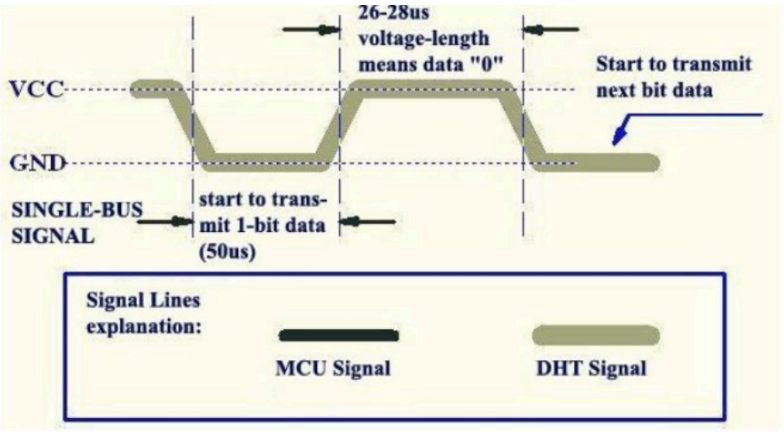


Dht11의 타이밍 다이어그램을 참고하여 **FSM**을 구현하였다.

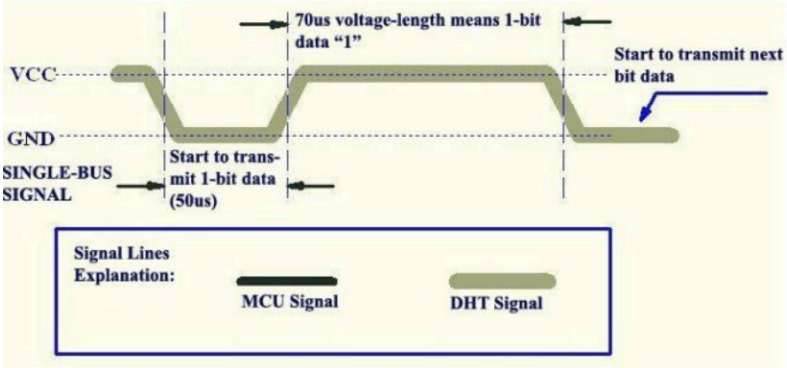
초기 **IDLE** 상태에서 **MCU(**현재 시스템에서는 **FPGA)**가 **data**라인을 **VCC**로 유지하고 있는다. 시작을 위해선 **LOW**로 일정시간**(18ms** 이상**)** 유지해줘야하며 이후 **20~40us**동안 **DHT**의 응답을 기다린다.

이후 제어권이 **DHT11** 에게로 넘어가며 **SYNC**를 맞추는 구간을 지나고 **DATA**를 전송하는 구간으로 넘어가게 된다.

타이밍 다이어그램을 보면 고려해야할 부분이 생긴다 **WAIT** 구간에서 **data line** 의 제어권이 **mcu -> dht** 로 넘어갈 때 언제 제어권을 넘겨줄 것인가 하는 문제이다.



Vcc를 26~28us 유지하면 0



Vcc를 70us를 유지하면 1

해당 그림은 센서가 **0** 과 **1**을 보내는 방식에 대한 설명을 하는 표이다.
그림에서는 **high** 를 **26~28 us** 를 유지하면 **0**으로 읽고 **70us**를 유지하면 **1**로 읽으라고 설명한다.
그러나 이 부분도 고려해야할 부분이 있다.
28~69 us 까지는 어떻게 읽어야 하는 가에 대한 설명은 되어 있지않다. 위에서 설명한 데이터 라인 제어권에 관한 내용과 **0,1**을 어떻게 인식할 것인가에 대한 내용은 아래 코드를 설명하며 다룬다.

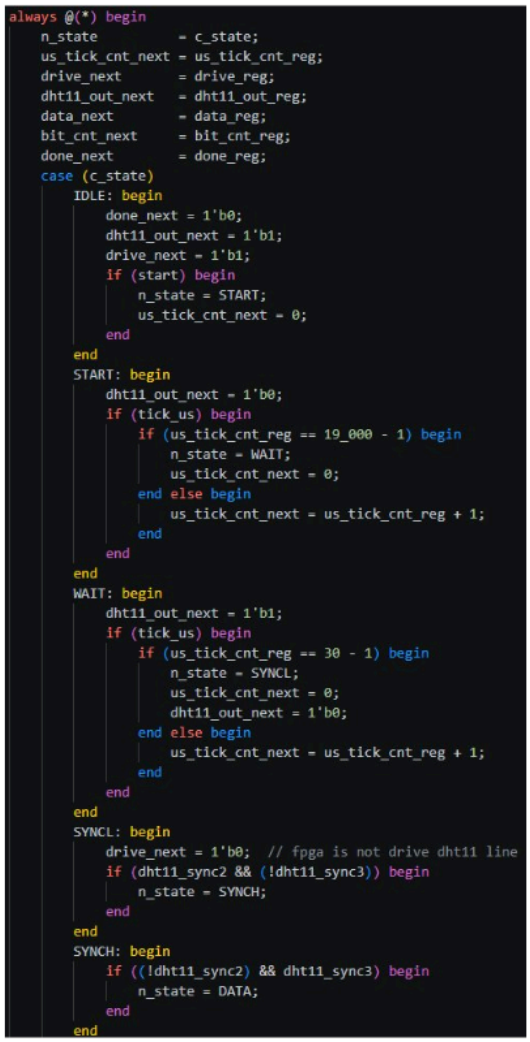
```
always @(posedge clk, posedge rst) begin
    if (rst) begin
        dht11_sync1 <= 0;
        dht11_sync2 <= 0;
        dht11_sync3 <= 0;
    end else begin
        dht11_sync1 <= dht11;
        dht11_sync2 <= dht11_sync1;
        dht11_sync3 <= dht11_sync2;
    end
end
```

Edge detecting 및 신호 안정화를 위한 F/F

```
always @(posedge clk, posedge rst) begin
    if (rst) begin
        c_state         <= IDLE;
        us_tick_cnt_reg <= 0;
        drive_reg       <= 1'b1;
        dht11_out_reg   <= 0;
        data_reg        <= 0;
        bit_cnt_reg     <= 0;
        done_reg        <= 0;

    end else begin
        c_state         <= n_state;
        us_tick_cnt_reg <= us_tick_cnt_next;
        drive_reg       <= drive_next;
        dht11_out_reg   <= dht11_out_next;
        data_reg        <= data_next;
        bit_cnt_reg     <= bit_cnt_next;
        done_reg        <= done_next;
    end
end
```

State 업데이트 블록



Latch 방지를 위한 초기화

초기 통신 라인 high를 위한 대기 블록

Dht11의 trigger 신호 활성화 블록

Dht11의 응답 대기 블록

Dht11의 응답, edge detecting

```
DATA: begin  // 100Mhz
    if (dht11_sync2 && (!dht11_sync3)) begin  // 상승엣지
        us_tick_cnt_next = 0;
    end else if ((!dht11_sync2) && dht11_sync3) begin // 하강엣지
        us_tick_cnt_next = 0;
        if (us_tick_cnt_reg < 5_000) begin
            data_next = {data_reg[38:0], 1'b0};
        end else begin
            data_next = {data_reg[38:0], 1'b1};
        end
        if (bit_cnt_reg == 40 - 1) begin
            n_state = STOP;
            bit_cnt_next = 0;
        end else begin
            bit_cnt_next = bit_cnt_reg + 1;
        end
    end else if (dht11_sync3) begin
        us_tick_cnt_next = us_tick_cnt_reg + 1;
    end
end
STOP: begin
    done_next = 1'b1;
    if (tick_us) begin
        if (us_tick_cnt_reg == 50) begin
            n_state = IDLE;
            us_tick_cnt_next = 0;
        end else begin
            us_tick_cnt_next = us_tick_cnt_reg + 1;
        end
    end
end
```

Receive data, edge detecting

종료 50us 대기 후 IDLE

```
// ----------------------------------------------
// check sum
// ----------------------------------------------
always @(posedge clk, posedge rst) begin
    if (rst) begin
        valid_reg <= 0;
    end else begin
        if (done_reg) begin
            if (rx_data == com_data) begin
                valid_reg <= 1'b1;
            end else begin
                valid_reg <= 1'b0;
            end
        end
    end
end
assign valid = valid_reg;
assign humidity =(valid) ? humidity_reg : 0;
assign temperature = (valid) ? temperature_reg : 0;
```

Check sum & output

해당 온습도 측정 센서는 입력으로 **40bit**를 주며 **32bit**가 온습도 데이터 **8bit**가 **check sum** 을 위한 데이터이다**. FSM**으로 구현하였으며**, 1-wire** 통신방식을 사용해 데이터라인 제어권을 어떻게 안정적으로 넘길지 그리고 **0**과 **1**을 어떻게 구분할지 고려하였다**.**

우선 초기상태**(IDLE)**에서 데이터라인**(dht11)**에 **1**을 입력해 **high** 를 유지하고 있다.
Start 라는 신호가 입력되면 즉시 **START state** 로 업데이트 되어 데이터라인을 **0**으로 **19ms(**최소 **18ms** 에대한 **1ms** 마진**)**를 유지한 뒤 **WAIT state** 로 넘어간다.
WAIT은 **30us**를 유지한 뒤 **SYNCL state** 로 넘어가 제어권을 **dht11**로 넘겨준다**. 30us**로 제어권을 넘겨주는 시간을 설정한 이유는 너무 늦게 넘겨주게되면 **mcu**와 **dht**가 한개의 라인을 서로 제어해 문제가 생길 수 있기 때문이다**.** 따라서 **20us~40us** 사이인 **30us**구간을 안정화된 구간이라고 판단하여 설정했다**.**

SYNCL은 데이터라인의 상승 엣지를 **catch** 하여 **SYNCH state**로 천이하며**, SYNCH**는 데이터라인의 하강엣지를 **catch** 해 **DATA state**로 천이 한다**.**

DATA state는 **0**과 **1**을 구분하는 방식이 데이터라인을 **high(vcc)**로 얼마동안 유지하느냐이기 때문에 **, low** 구간과 **high** 구간을 모두 감시하는 대신 상승엣지를 **detecting** 하여 그 시점부터 카운트를 시작하고**,** 하강엣지가 검출되는 시점에 그동안 누적된 카운트 값을 확인하여 비트를 판별하는 방식으로 설계하였다**.** 이때 **0**과 **1**을 구분짓는 기준 시간을 **50us**로 설정하여**, high**구간이 이보다 짧으면 **0,** 길면 **1**로 판별하였다**.** 온습도 데이터 **32bit**와 **checksum**데이터 **8bit**를 받기 위해 **DATA state**에 **bit counter**를 두어 **40**번 반복하여 데이터를 수신하고 종료될 수 있게 하였다**.** 데이터를 받는 방식은 **DHT11**이 데이터를 **MSB**부터 보내주기 때문에**,** 판별된 비트를 **LSB**위치에 입력한 뒤 계속 왼쪽으로 **bit shift**하여 최종적으로 **MSB**까지 채워지는 방식으로 설계하였다**.**

STOP state 에서는 **dht**가 **50us**동안 **low**로 **data line** 을 유지하고 제어권을 놓으며 다시 **mcu** 가 가져와야 하기에 **50us** 를 기다린 뒤 **IDLE state** 에서 제어권을 가져온다**.**

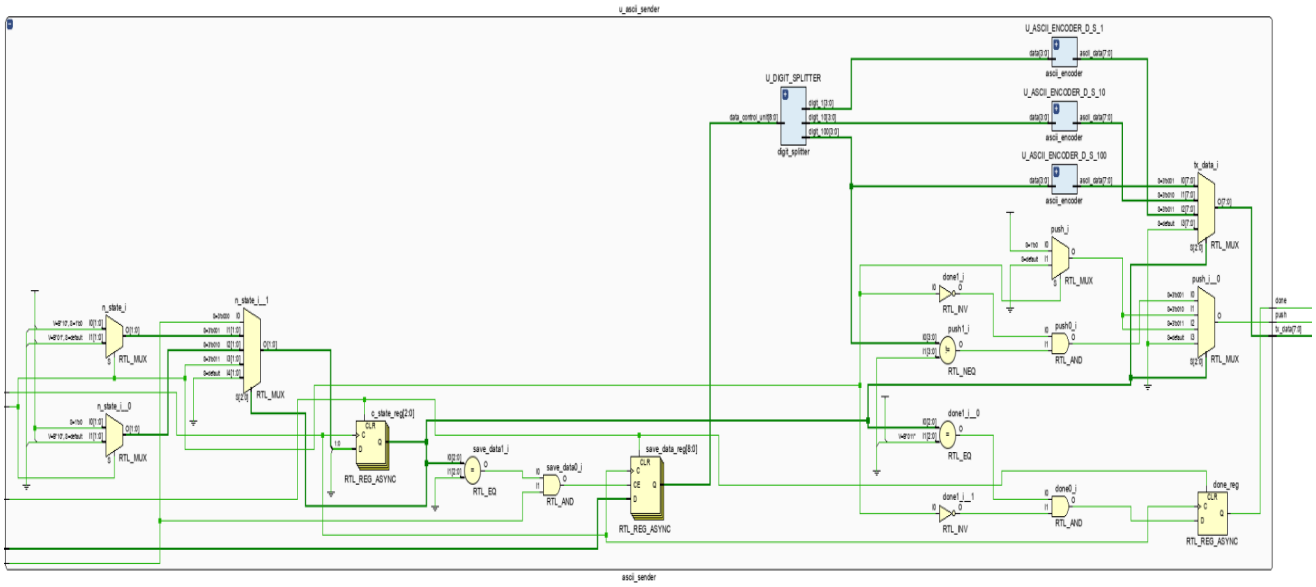
1.3.6. Sub block #6 sender

Ascii sender는 FPGA 내부 컨트롤러가 계산한 숫자 데이터를 사람이 읽을 수 있는 아스키 문자로 변환한 뒤, UART TX를 통해 PC 터미널로 순차 송신하는 출력 제어 모듈입니다.

센서 측정값이나 스톱워치, 워치 결과와 같은 데이터가 입력되면, 모듈은 먼저 digit_splitter을 이용해 해당 숫자를 백의 자리, 십의 자리, 일의 자리 등으로 각각 쪼갭니다.

또한 Ascii encoder을 통해 데이터 및 입력 신호를 아스키 코드로 변환하여 PC로 전송합니다. UART TX는 한 번에 1바이트씩만 전송할 수 있기 때문에, 내부 상태 머신(FSM)이 TX 모듈의 바쁜 상태(tx_busy)를 체크하면서 이 문자들을 한 글자씩 순서대로 밀어 넣어 화면에 출력되도록 제어합니다.

Ascii sender schematic



Ascii sender구조 코드설명

```
localparam [2:0] IDLE = 0, SEND_100 = 1, SEND_10 = 2, SEND_1 = 3;
reg [2:0] c_state, n_state;

//ascii_sender 입력 시 신호가 연속으로 들어와도 겹치지 않도록 idle,send_data가 1일때만 새로운 데이터가 들어왔다고 아는것
always @(posedge clk, posedge reset) begin
    if (reset) begin
        save_data <= 9'd0;
    end else if (c_state == IDLE && send_start) begin
        save_data <= selected_data;
    end
end


// current_state
always @(posedge clk, posedge reset) begin
    if (reset) begin
        c_state <= IDLE;
        done <= 1'b0;
    end else begin
        c_state <= n_state;
        done <= (c_state == SEND_1) && !full;
    end
end
```

sender 상태 지정

현재상태 지정

```
// next_state
always @(*) begin
    n_state = c_state;
    case (c_state)
        IDLE: begin
            if (send_start) begin
                n_state = SEND_100;
            end else begin
                n_state = IDLE;
            end
        end
        SEND_100: begin
            if (!full) begin
                n_state = SEND_10;
            end else begin
                n_state = SEND_100;
            end
        end
        SEND_10: begin
            if (!full) begin
                n_state = SEND_1;
            end else begin
                n_state = SEND_10;
            end
        end
        SEND_1: begin
            if (!full) begin
                n_state = IDLE;
            end else begin
                n_state = SEND_1;
            end
        end
        default: n_state = IDLE;
    endcase
end
```

IDLE

SEND_100 (백의자리)

SEND_10 (십의자리)

SEND_1 (일의자리)

```
// output logic
always @(*) begin
    push = 1'b0;
    tx_data = 8'h00;
    //busy = (c_state != IDLE); //register추가하고 신호 무시되지 않도록 busy신호추가
    case (c_state)
        SEND_100: begin
            tx_data = w_digit_100;
            if (!full && digit_100 != 0)
                push = 1'b1;  // full이 아닐 때만 push 발생
        end
        SEND_10: begin
            tx_data = w_digit_10;
            if (!full) push = 1'b1;  // full이 아닐 때만 push 발생
        end
        SEND_1: begin
            tx_data = w_digit_1;
            if (!full) push = 1'b1;  // full이 아닐 때만 push 발생
        end
        default: begin
            push = 1'b0;
            tx_data = 8'h00;
        end
    endcase
end
endmodule
```

FULL = 0일때 계속 PUSH

```
module ascii_encoder (
    input [3:0] data,
    output reg [7:0] ascii_data
);

    always @(*) begin
        case (data)
            4'd0: ascii_data = 8'h30;  //0
            4'd1: ascii_data = 8'h31;  //1
            4'd2: ascii_data = 8'h32;  //2
            4'd3: ascii_data = 8'h33;  //3
            4'd4: ascii_data = 8'h34;  //4
            4'd5: ascii_data = 8'h35;  //5
            4'd6: ascii_data = 8'h36;  //6
            4'd7: ascii_data = 8'h37;  //7
            4'd8: ascii_data = 8'h38;  //8
            4'd9: ascii_data = 8'h39;  //9
            4'd10: ascii_data = 8'h3A;  // :
            default: ascii_data = 8'h30;
        endcase
    end

endmodule
```

아스키코드에 맞는 데이터값 대입

```
module digit_splitter (
    input  [8:0] data_control_unit, // control_unit에서 받는 신호, 센서거리때문에 9비트
    output [3:0] digit_1,
    output [3:0] digit_10,
    output [3:0] digit_100
);

    assign digit_1   = data_control_unit % 10;
    assign digit_10  = (data_control_unit / 10) % 10;
    assign digit_100 = (data_control_unit / 100) % 10;

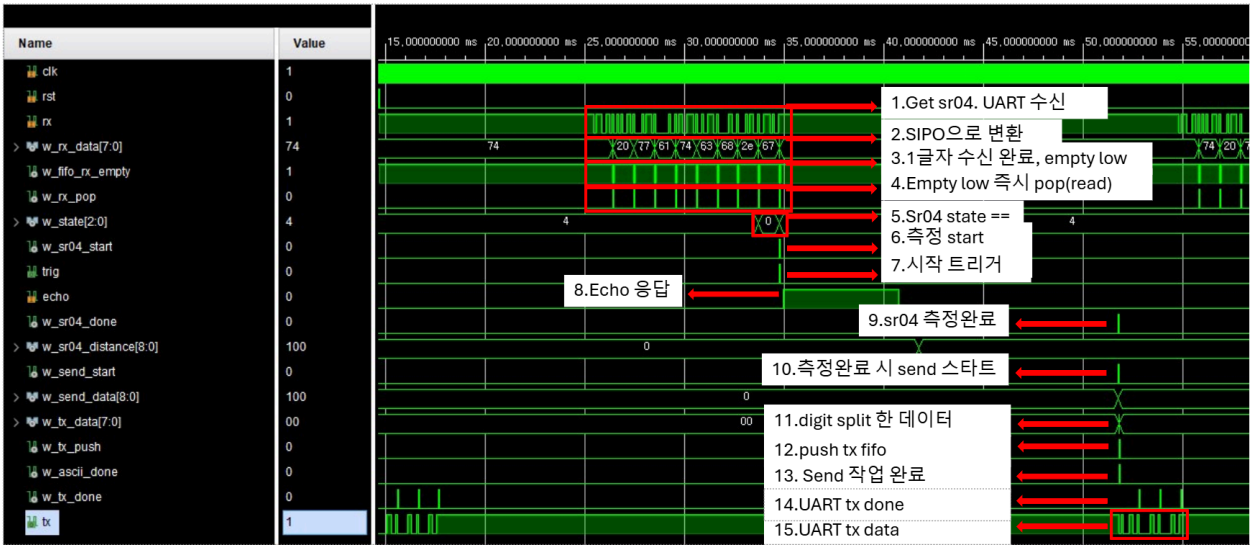
endmodule
```

연산자를 사용하여 백의자리,십의자리 ,일의자리 계산함

2. Simulation

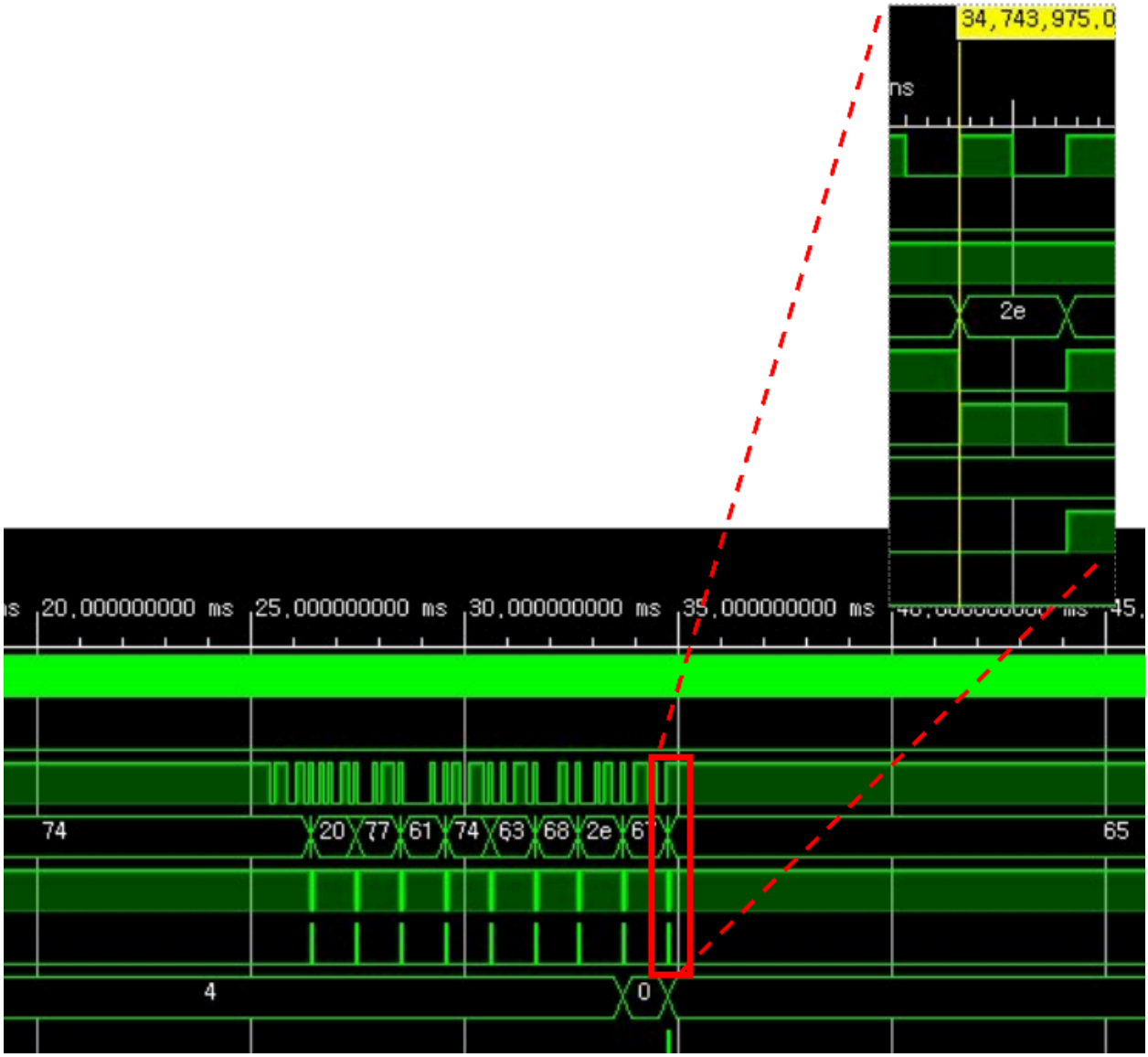
시뮬레이션은 sr04의 거리 데이터 수집을 시뮬레이션 하여 동작을 검증한다.

2.1. Top Simulation

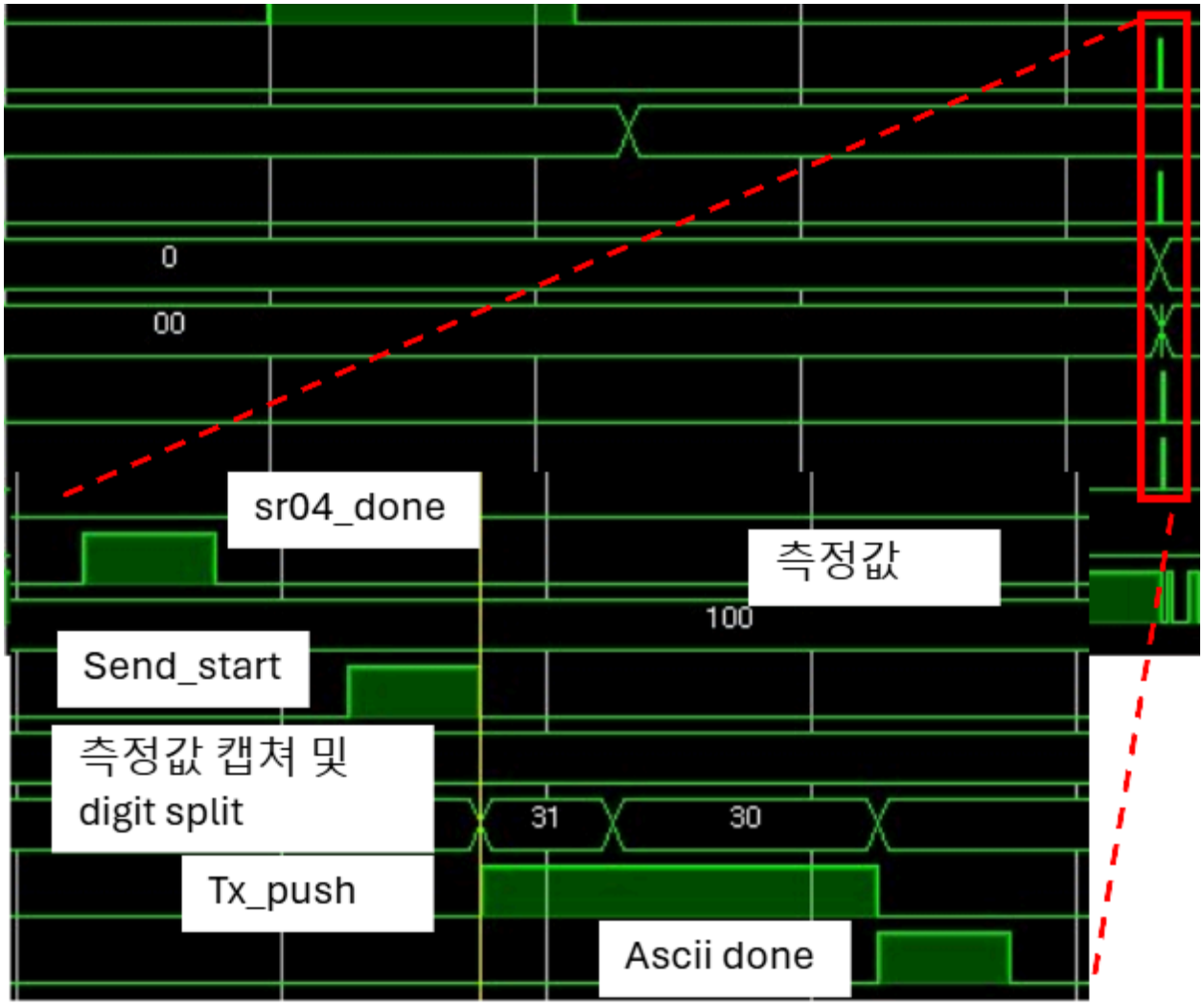


해당 시뮬레이션은 get sr04. 이라는 커맨드를 uart로 입력 받아 센서데이터 측정 후 uart를 통해 pc까지 전송하는 과정을 시뮬레이션한다.

1. uart + fifo를 통해 rx데이터를 송신받고 Serial In Parelle Out 방식으로 변환한다.
2. ascii decoder에서 empty 신호를 통해 rx fifo에 데이터가 있음을 확인하고 즉시 pop(read)을 진행한다.
3. “.” 을 기준으로 명령어의 종료를 읽어오며 “.”의 hex 는2e이다.



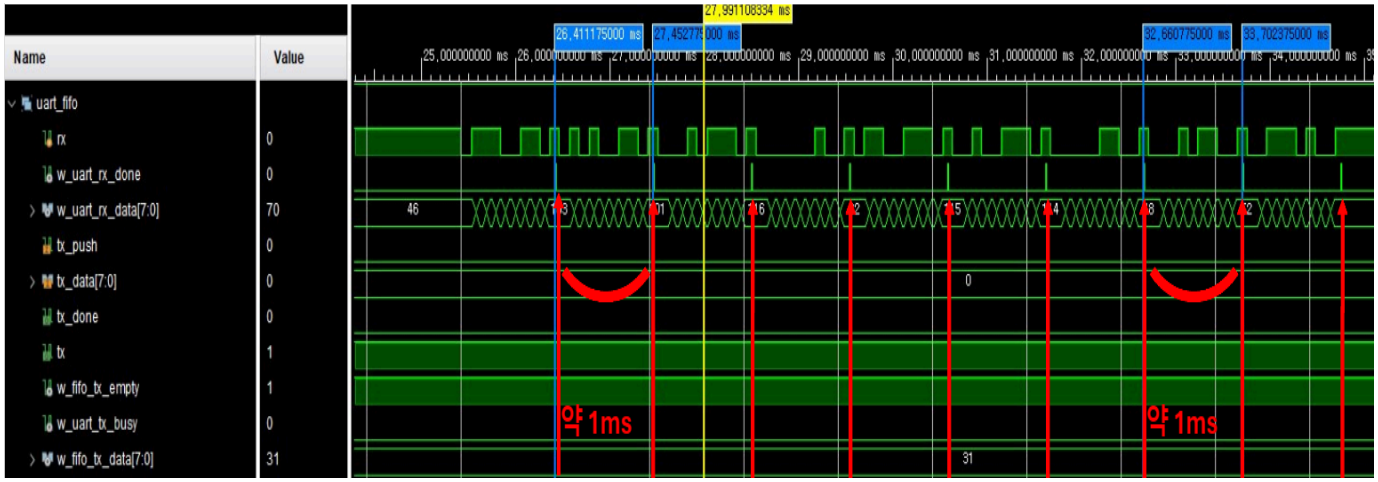
4. Ascii decoder는 2e가 입력될 때까지 값들을 축적하며 2e가 들어오는 시점에 축적한 값에 해당하는 명령을 controller 로 전송한다.
5. Controller 는 명령을 구분하여 해당 모듈(watch,stopwatch, dht11,sr04)에 정보를 가져올 수 있는 신호를 전달한다.(sr04의 경우 측정 start 신호)
6. Start 신호를 받은 sr04 controller 는 측정을 위한 시퀀스를 시작한다. (idle -> trigger -> wait -> echo -> calc -> busy)
7. sr 04 controller 는 시퀀스 완료 후 done 신호를 출력하며 이때 contoller 는 측정 값을 받아와 sender 에게 전달한다.



8. Sender 에게 send start 신호를 전달한다.
9. Sender 는 start 신호를 입력받고 측정값을 캡쳐하여 digit split 및 ascii 값으로 변환하여 fifo에 푸쉬하고 완료하면 ascii done 을 띄운다.
10. 이후 uart + fifo는 tx 시퀀스를 진행한다.

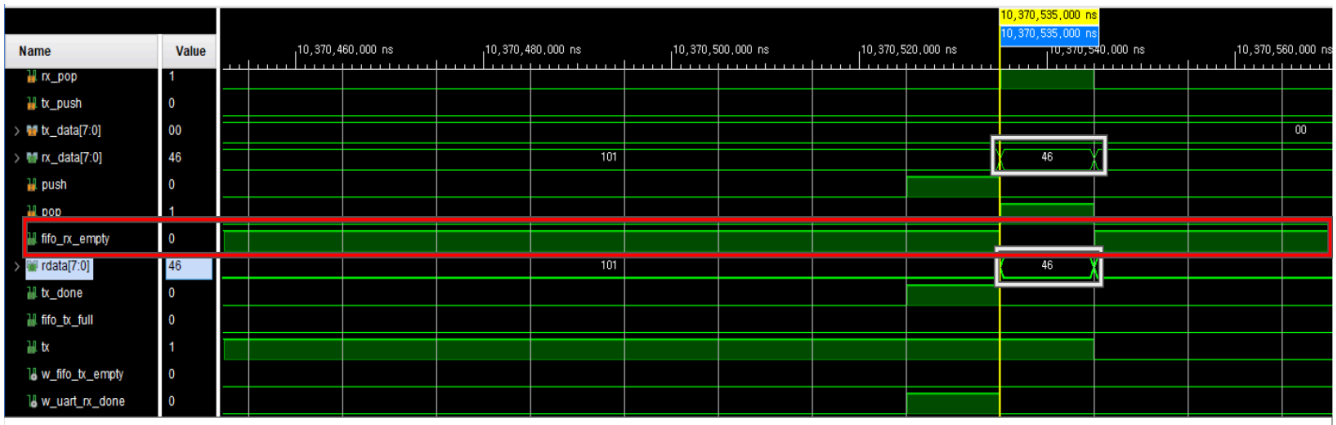
2.2 Submodule Simulation

2.2.1 Submodule #1 Tick_generator



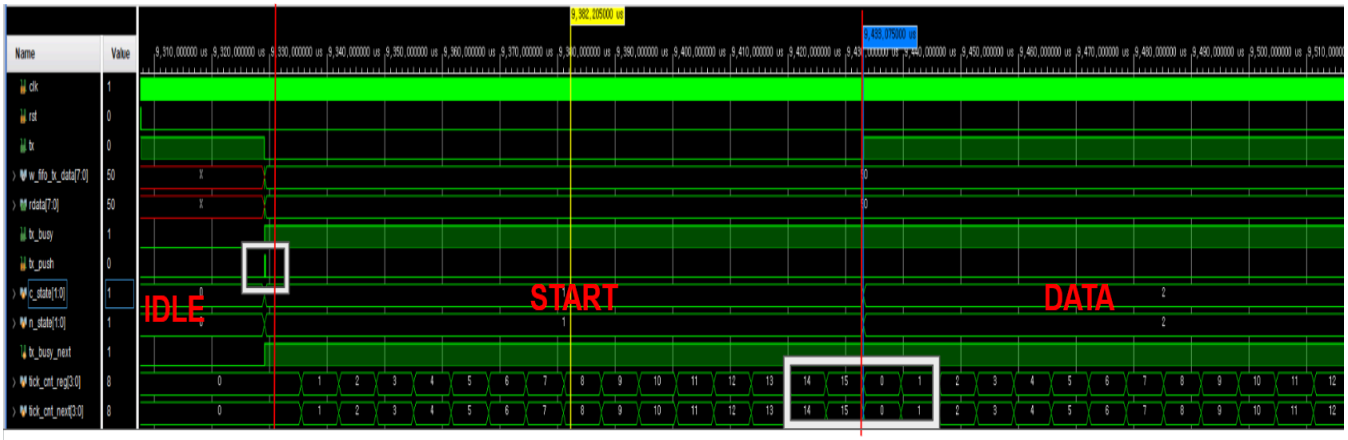
9600 baud에서는 한 비트를 보내는 데 약 104us가 걸리고, 한 글자는 총 10비트로 전송되기 때문에 글자 하나를 보내는 데 약 1.04ms가 걸린다.

2.2.2 Submodule #2 UART_FIFO



1. UART RX을 통해 rx_data에 46이 들어옴
2. push = 1이 되어 데이터가 FIFO에 저장되고 다음 클럭에 fifo_rx_empty = 0으로 바뀌어 데이터가 있는것을 확인
3. pop = 1이 되어 데이터를 읽음
4. rdata출력에 저장된 데이터가 정상적으로 출력
5. 데이터를 읽은 후 fifo_rx_empty = 1이 되어 FIFO가 비어있음을 확인

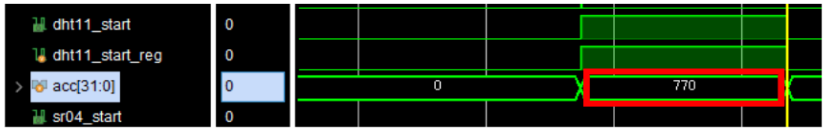
2.2.3 Submodule #3 UART_FIFO

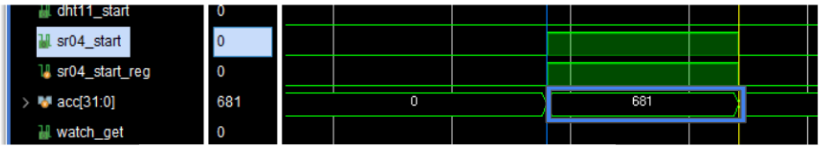


1. START, DATA 상태에서 TICK 16까지 카운트되는것을 확인
2. IDLE -> START -> DATA -> STOP 순서대로 넘어가는것을 확인
3. TX = 0일 때 IDLE -> START로 넘어가는 것을 확인
4. IDLE -> START 넘어간 후 tx_busy = 1이 올라온 것을 확인
5. tx_busy = 1일 때, tx_push = 1이 뜨는것을 확인

2.2.4 Submodule #4 Ascii_decoder

```
32'd770 : begin // get dht11
    if (cnt == 810) begin
        dht11_start_reg = 1'b0;
    end else begin
        dht11_start_reg = 1'b1;
    end
end
32'd681 : begin // get sr04
    if (cnt == 810) begin
        sr04_start_reg = 1'b0;
    end else begin
        sr04_start_reg = 1'b1;
    end
end
32'd1250 : begin // rs stopwatch
    if (cnt == 810) begin
        rs_stopwatch_reg = 1'b0;
    end else begin
        rs_stopwatch_reg = 1'b1;
    end
end
```

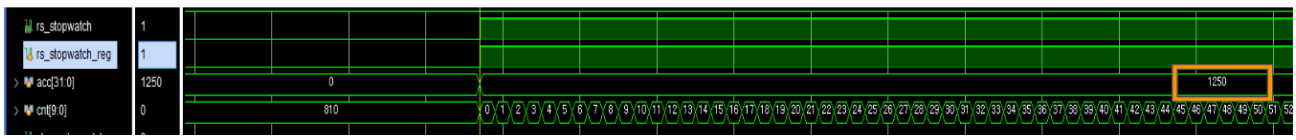




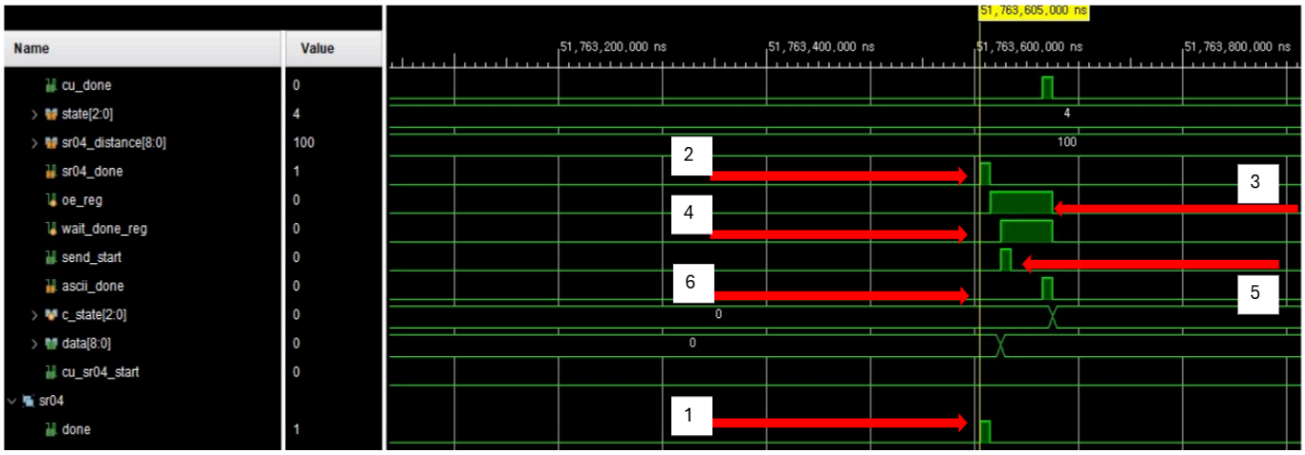
get dht11. : 온습도센서 값 불러오기

get sr04. : 거리센서 값 불러오기

rs stopwatch. : 스톱워치 시간 불러오기



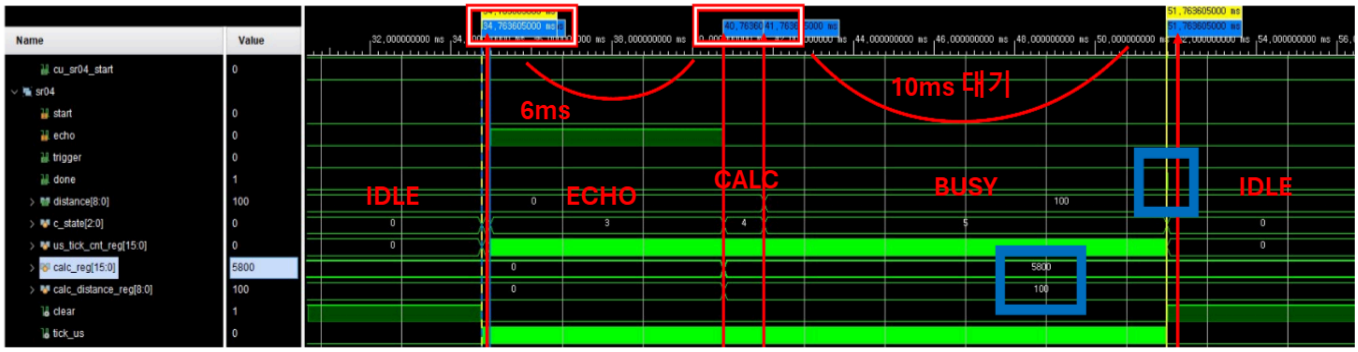
2.2.5 Submodule #5 Control_unit



Control_unit(동작 상황 :SR04 sensor -> Control_unit)

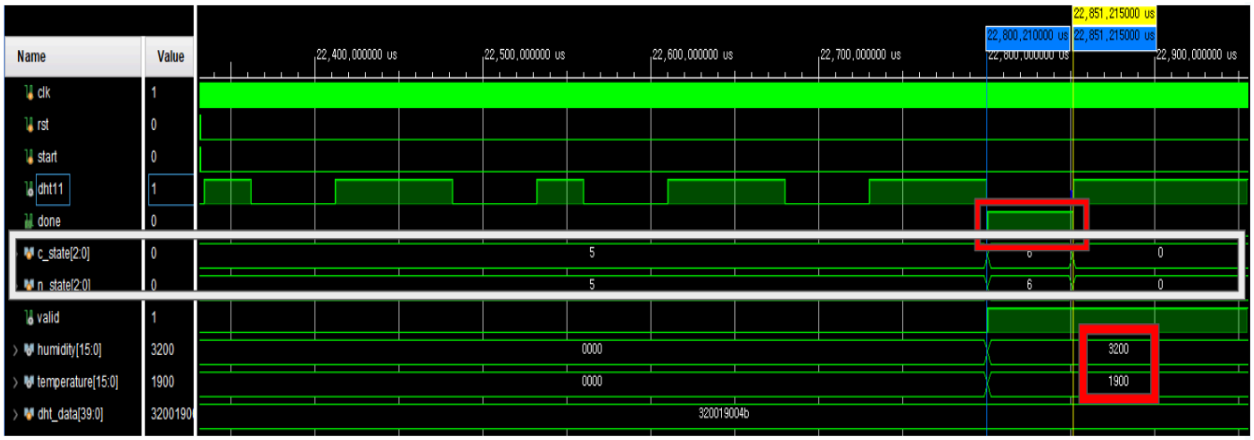
1. SR04_sensor에서 done 신호를 입력받는다.
2. Control_unit의 SR04_done 신호를 high로 띄운다.
3. oe_reg(output enable)신호 high로 올라온다.
4. wait_done_reg = 1(Ascii_sender 작업중)
5. send _start 신호 high(Ascii_sender 작업 시작)
6. ascii_done = 1(Ascii_sender 작업 완료)

2.2.6 Submodule #6 SR04 sensor



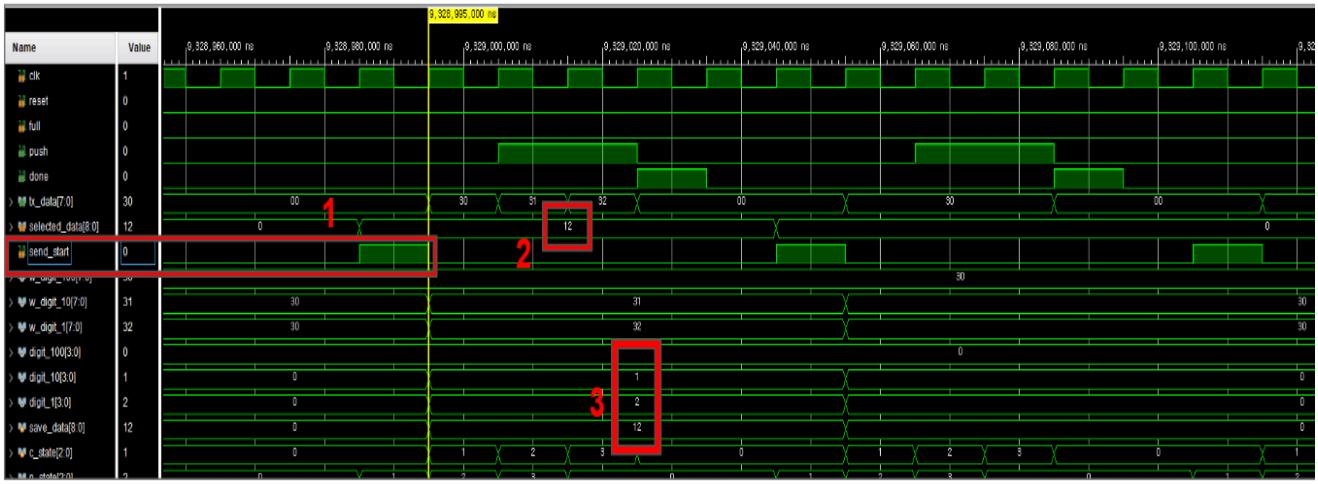
1. Echo = 1이 되는 순간 ECHO(state:3)상태로 전이되고 us_tick_cnt_reg값이 1씩 증가하며 echo가 high로 유지되는 시간을 카운트 한다.(약 6ms동안 유지)
2. Echo = 0으로 떨어지면 측정된 시간을 calc_reg에 저장한다.
3. Formula공식을 활용하여 거리를 계산한다.(state:4)
4. 계산이 완료되면 BUSY상태로 넘어가고(state:5) 10ms 동안 대기한다.
5. BUSY상태에서 IDLE상태로 넘어갈때 done = 1이 되어 측정완료신호를 알린다. (calc_distance_reg = 100cm)

2.2.7 Submodule #7 DHT11 sensor



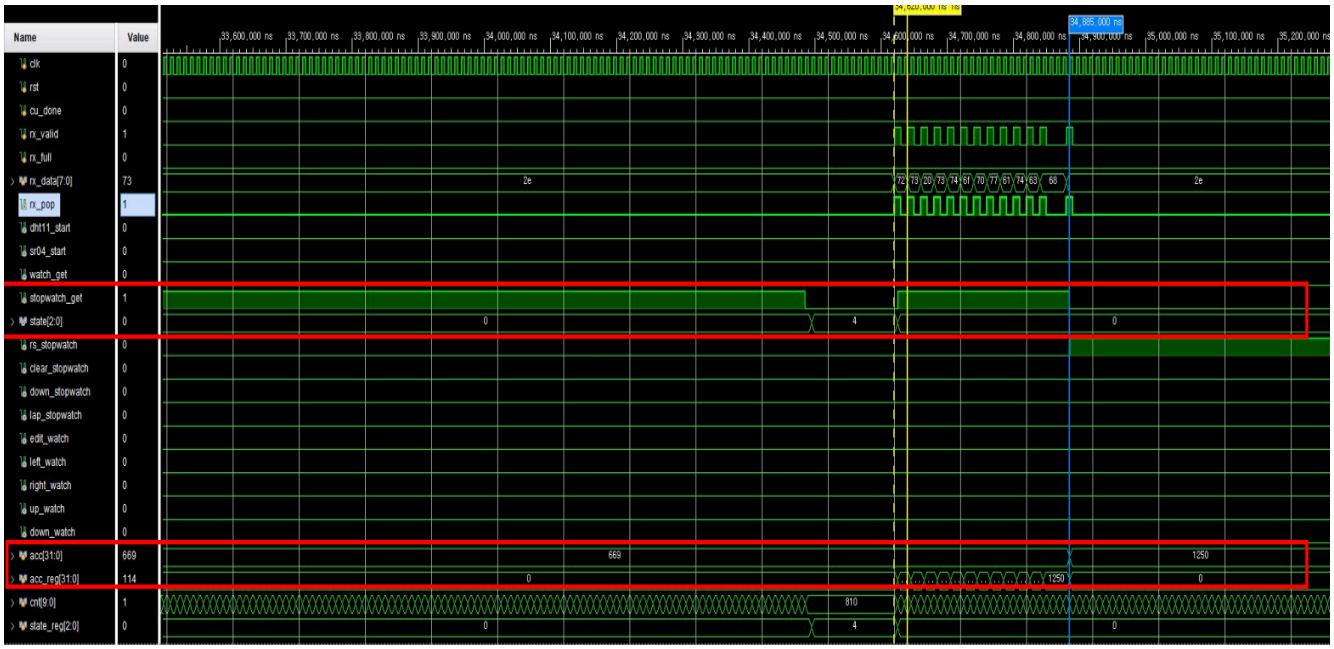
1. 데이터 수신 완료 후, done = 1이 되는 동시에 상태가 5(DATA)에서 6(STOP)으로 넘어간다.
2. humidity : 16'h3200 : 50, temperature : 16'h1900 : 25의 습도,온도값이 나온다.
3. valid = 1로 된 것은 데이터 수신이 잘 되었음을 알린다.
4. dht11_data에서 마지막 4b값은 습도와 온도 데이터의 합인 50 + 0 + 25 + 0 = 75이며, 16진수 4B와 일치하여 데이터가 정상적으로 수신되었음을 확인할 수 있다.

2.2.8 Submodule #8 Ascii_sender



1. send_start = 1이 되면 데이터값이 들어옴
2. selected_data = 12가 들어오면 사라지지 않도록 save_data 12 저장
3. w_digit_1 = 2, w_digit_10 = 1에 각 자릿수에 맞게 데이터값이 들어감 w_digit_100 = 0(12)

3. Trouble shooting



현상

ASCII decoder가 UART로 수신한 명령어("rs stopwatch")를 처리하는 과정에서, `stopwatch_get` 신호가 다음 신호를 받을 때까지 한번 떨어지고 유지되는 현상이 발생함. `Acc`가 초기화가 되지 않는 것이 확인되어, 초기화 구문이 제대로 동작하지않는 것이 의심됨.

관찰 근거

1. 이전 stopwatch_get의 신호가 다음 명령어를 받는 순간에 한번 끊겼다 유지됨.

2. 카운트는 정상적으로 810ms유지하고 IDLE == 4를 유지하고 있으나 다음 카운터가 초기화 된 뒤 다시 stopwatch_get이 1로 유지됨

3. 명령어 해석을 acc로 처리하고 있었기 때문에 acc의 값의 변화가 다음명령어를 받을 때까지 유지되어 나온다고 추측함.

원인 분석

ASCII decoder 내부에서 명령어 로직이 누적된 값을 참조하고 있음 — acc는 클럭에 의해 저장되는 순차 논리 신호이다. acc가 명령어 판별을 할 때 축적된 값을 이용하지만 신호를 일정시간 다 보낸 뒤에 초기화가 이루어져야했다. 그러나 확인 결과 acc 의 값이 초기화가 이루어 지지않았고 값이 변하는 시점인 다음 명령어까지 이전 축적된 값을 유지하고있어 해당 명령어를 받을 때 카운터가 초기화 되어 다시 신호가 high 올라온다고 판단했다.

해결 방안

명령어 수행 종료 시점(카운터 == 810)에 acc를 0으로 초기화하여 이전 명령어의 누적된 값이 다음 명령어 처리에 영향을 주지 않도록 수정하였다. 이를 통해 새로운 명령어 수신 시 항상 초기화된 상태에서 시작되도록 하여 명령어의 동작이 정상적으로 동작하도록 해결하였다.

```
always @(posedge clk, posedge rst) begin
if (rst) begin
    acc_reg   <= 0;
    acc       <= 0;
    state_reg <= 3'd4;
end else if (cu_done) begin
    acc_reg   <= 0;
    acc       <= 0;
    state_reg <= 3'd4;
end else begin
    if (cnt == 810) begin
        acc <= 0;
    end
```

acc<=0; 추가

4.Further work

본 프로젝트에서는 UART 텍스트 커맨드를 이용해 초음파, 온습도, 시계/ 스톱워치 기능을 제어하는 FSM기반 시스템을 구현하였다.

추후 개선 방향으로 다음 2가지를 제시한다

1. 에러 처리 및 예외 상황 대응 미비

현재 구조는 명령어 오타나 비정상 입력에 대한 대응이 빠져있다. 예를 들어 사용자가 “get sr04.” 대신 “get sr0.” 처럼 오타를 입력하면 시스템은 이를 무시할 뿐, 오류 상태임을 알려주지 않는다. 또한 “get sr04” 처럼 “.”을 빠트리는 상황이 올 경우 사용자는 무엇이 잘못됐는지 모를 수 있다. 이 두 경우 사용자에게 에러를 알려주며 타임아웃 기반 보호 로직을 추가하면, 시스템이 잘못된 입력상태에 계속 머무르는 것을 방지하고 이용 편의성을 향상시킬 수 있다.

2. FSM 구조의 확장성

현재 FSM은 각 명령어를 문자 단위로 순차적으로 비교하여 명령어를 판별하는 구조로 설계하였다. 향후 명령어가 증가할 경우 상태 수와 비교 조건이 함께 증가할 수 있으므로, 명령어 수신과 명령어 판별 기능을 분리하여 FSM의 복잡도를 줄이는 방향으로 개선할 수 있다. 이를 통해 기능 추가에 따른 설계 변경을 최소화하고 코드의 가독성과 유지보수성을 향상시킬 수 있다.

5. Project video

과제파일과 함께 제출